Multi-Drone Cooperative Search with Random Faults

A fault-tolerant cooperative multi-agent system trained with deep reinforcement learning, analysed through the multi-agent paradigm

Simone Rimondi

Student ID 0001189110

simone.rimondi5@studio.unibo.it

Multi-Agent Systems

Master's Degree in Artificial Intelligence, University of Bologna

Academic Year 2025/2026

Abstract

We study cooperative target search by a team of drones on a partially observable grid, under limited communication, a fixed time budget and — the defining extension of this work — the possibility that any drone permanently breaks at any moment, with no central authority to announce the loss. A single Dueling Double DQN with parameter sharing drives every drone from strictly per-drone inputs, batched into one forward pass only for speed; per-episode domain randomization and a compact context vector turn that one policy into a generalist across team size, sensing, communication range, obstacle density, and fault rate. Fault knowledge is fully decentralized: a wreck emits only a passive distress beacon, and crash information spreads through the same range-limited map fusion as any other observation, so each drone acts on a sound-but-possibly-stale belief about how many teammates remain. We read the resulting system through the multi-agent paradigm, namely coordination without protocols, environment-mediated (stigmergic) dispersion, an artefact and autonomy analysis, and an epistemic account of fault handling, and we evaluate it empirically. The single policy generalizes across every randomization axis, including *beyond* its trained ranges on team size, vision and communication; most importantly, it degrades *gracefully* under faults: across the entire trained fault range success drops by under six points while the team loses, on average, more than half a drone, and even at more than triple the trained fault rate it still completes two missions in three, with no cliff. We close with an ethical and regulatory analysis (GDPR, the EU AI Act and its military carve-out, and the autonomous-weapons scenario) that takes seriously the dual-use nature of a capability whose fault tolerance would make an attack swarm as resilient as a rescue one.

GitLab repository	https://dvcs.apice.unibo.it/pika-lab/courses/ai-ethics/projects/rimondi2526-mas
GitHub repository	https://github.com/simoswish02/Multi-Agent-Systems-Project
Trained weights	https://huggingface.co/simoswish/MAS_MultiDroneExploration
Video walkthrough	https://youtu.be/az1LtAcZYZo

Contents

1 Introduction

When something must be found quickly over a large area, be it a missing hiker, survivors in a collapsed structure or the front of a spreading wildfire, teams of small autonomous aircraft are increasingly the tool of choice: they are cheap, expendable, fast to deploy and, above all, *parallel* [25, 42]. Yet the hard problem in multi-drone search is not flight; it is coordination under scarcity. Each vehicle perceives only a small neighbourhood of a large, unknown environment; communication reaches only nearby teammates, so no shared global picture exists; and the mission clock is unforgiving, since in search and rescue time *is* the objective. A team that cannot divide the search space effectively degenerates into an expensive single searcher.

This project adds to that already delicate setting the ingredient that real operations always contain and idealized models usually omit: **failure**. Small drones break: motors fail, batteries die, weather intervenes. And they break *mid-mission*, unpredictably, without ceremony. A search system that presumes a fixed team is brittle in exactly the situations that matter most. Worse, in a genuinely decentralized setting a drone's death is not even *announced*: its teammates must discover the loss through their own sensing and communication, and until they do, they coordinate with a phantom. Fault tolerance in a multi-agent system is thus not one problem but three: surviving the loss (redundancy), learning of it (decentralized fault knowledge), and adapting to it (re-planning the division of labour). This report treats all three as first-class design objects.

The problem. Concretely, we study the following task, formalized in Section 3. A team of N drones spawns co-located on a corner of a 32×32 occupancy grid scattered with obstacles; a single target, drawn from the region reachable from the reference corner, is hidden at an unknown cell. Each drone sees only a window of radius r_{vis} around itself, remembers what it has seen, and fuses its accumulated map with any teammate within communication range r_{comm}. The team has a hard budget of $T_{\max}$ turns to physically reach the target. At every turn, any live drone may irreversibly break with probability p_{fault}; a wreck stops sensing, moving and communicating, its turns still consume the budget, and no one is notified: fault knowledge spreads only through a passive distress beacon and the ordinary map fusion. Success is binary and team-level: *some* drone reaches the target in time, or the episode fails.

The approach. We solve the task with cooperative deep reinforcement learning under parameter sharing [17]: a single Dueling Double DQN [33, 54, 56] drives every drone: it consumes that drone's private maps through a convolutional architecture with an attention bottleneck, and is trained on the pooled experience of the whole team. Rather than fixing one operating condition, every training episode randomizes the regime [51], sampling the vision radius, the communication range, the team size, the obstacle density and the fault probability. The sampled parameters that a real drone could plausibly know are fed to the network as a context vector, together with two per-drone quantities: an identity, which breaks the symmetry of parameter sharing and lets co-located drones adopt different roles from the first step, and the drone's own *belief* about how many teammates remain operational. The result is a single generalist policy that searches alone or in a team, myopic or far-sighted, in empty fields or cluttered mazes, and that experiences teammate death, and the need to absorb it, as an ordinary feature of its world. The fault model demands equal care on the learning side, since a fault is exogenous bad luck rather than a consequence of behaviour, and Section 5 details how the learning updates are made to respect that.

Why this is a multi-agent systems study. A reinforcement-learning pipeline, however carefully engineered, is not by itself the point of this report. The system is deliberately read, in Section 6, through the conceptual apparatus of multi-agent systems [24, 60]: the drones are audited against the defining properties of agenthood; the environment is analysed as a first-class abstraction that mediates and governs all interaction; the map fusion is read as knowledge-level,

protocol-free communication over a distributed shared dataspace; the visited-cell traces as a virtualized form of stigmergy; the fused map, the beacon and the action mask as coordination artefacts; the team's dispersion and post-fault *re-covering* (survivors sweeping a lost teammate's sector again) as emergent, self-organized structure that no component specifies; and the whole design is placed on the autonomy spectrum as operationally autonomous but motivationally not. This analysis is not decorative: it is what connects a concrete learned system to the questions about coordination, openness, resilience and responsibility that the multi-agent perspective exists to ask. The same lens then carries the final part of the report, where autonomy without deliberation and competence without transparency become ethical and legal problems rather than engineering trade-offs.

Contributions. The report makes the following contributions:

1. *A cooperative search environment with a decentralized fault model* (Sections 3–4): a configurable grid world, built from scratch, in which drone failures are permanent, unannounced, and discoverable only through a distress beacon and range-limited map fusion, no component ever accessing a global oracle of team state.

2. *A single generalist policy via domain randomization and context conditioning*: one shared network covering team sizes, sensing and communication regimes, obstacle densities and fault rates, with per-drone identity and believed-alive-count inputs enabling role differentiation and post-fault adaptation, trained under learning rules that treat a fault as exogenous bad luck rather than as a terminal state.

3. *A systematic multi-agent reading of the system* (Section 6): a detailed mapping between the implemented mechanisms and the paradigm's core concepts, namely agenthood, environment, coordination, artefacts, self-organization, openness, autonomy and multi-agent learning.

4. *An empirical demonstration of generalization and graceful degradation* (Section 7): a seeded, one-factor-at-a-time protocol (500 episodes per configuration) showing that the policy generalizes without a cliff across team size, vision and communication, including *beyond* the ranges it was trained on, while obstacle density is the one axis that genuinely governs difficulty; and, as the headline result, that success falls by under six points across the entire trained fault range and still reaches 66% at more than triple that rate, where half the team is lost on average, with no cliff.

5. *A prospective ethical and regulatory analysis* (Section 8): the capability's dual-use profile, its treatment under the GDPR and the EU AI Act (including the military carve-out that exempts precisely the gravest use), an argument that the system as designed could not lawfully or ethically serve as an autonomous weapon, and a set of design commitments that follow.

2 Background

This section reviews the concepts the rest of the report relies on: the value-based reinforcement-learning toolbox, its extension to multiple interacting learners, and the technique of domain randomization with context conditioning that lets a single policy serve many task variants. Readers familiar with deep value-based reinforcement learning may skim it and return to individual definitions as they are referenced later.

2.1 Value-based reinforcement learning

The standard formal model of an agent acting in an environment is the *Markov decision process* (MDP) [49], a tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ of states, actions, a transition kernel $P(s' \mid s, a)$, an expected

immediate reward $R(s,a)$ and a discount $\gamma \in [0,1)$. At each step the agent observes s_t, acts according to its *policy* $a_t \sim \pi(\cdot \mid s_t)$, receives r_t and moves to s_{t+1}; the quantity optimized is the expected *return* $G_t = \sum_{k\geq 0} \gamma^k r_{t+k}$. The Markov property, namely that the distribution of (s_{t+1}, r_t) depends on the history only through (s_t, a_t), is what licenses value functions defined on states rather than on histories. The action-value function of an optimal policy π^*, $Q^*(s,a) = \mathbb{E}_{\pi^*}[\, G_t \mid s_t = s,\ a_t = a\,]$, satisfies the Bellman optimality equation $Q^*(s,a) = \mathbb{E}[\, r + \gamma \max_{a'} Q^*(s',a') \mid s, a\,]$, and once Q^* is known acting optimally is trivial: $\pi^*(s) = \arg\max_a Q^*(s,a)$. *Q-learning* [57] estimates it from interaction alone, without knowledge of P or R, nudging the current estimate toward a bootstrapped target,

$$Q(s,a) \;\leftarrow\; Q(s,a) + \alpha\Big[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big],$$

and it is *off-policy*: the target uses the greedy action regardless of how a was chosen, so the behaviour policy may explore freely. The standard choice is *ε-greedy*, acting uniformly at random with probability ε and greedily otherwise, with ε annealed over training.

Tabular methods require one value per state–action pair, hopeless when states are high-dimensional (here, stacks of 32×32 spatial maps). *Function approximation* replaces the table with a neural estimator $Q_\theta(s,a)$ trained by regression on bootstrapped targets, a combination unstable in its naive form: consecutive samples are strongly correlated, and the target moves with every update of θ, since the same network appears on both sides of the Bellman equation. The *Deep Q-Network* (DQN) [33] stabilizes it with two mechanisms. *Experience replay* [28], a large circular buffer sampled uniformly at random, breaks temporal correlations and reuses each experience many times; a separate *target network* with frozen parameters θ^-, periodically synchronized with θ, supplies the bootstrap value and keeps the regression target fixed between synchronizations. The temporal-difference error is then minimized under a Huber loss. Three refinements are used in this work. *Double DQN* [54] removes the overestimation bias of a max operator that both selects and evaluates the next action with the same noisy estimator, by decoupling the two roles,

$$y \;=\; r + \gamma\, Q_{\theta^-}\big(s',\, \arg\max_{a'} Q_\theta(s',a')\big),$$

at negligible cost. The *dueling* network [56] splits the action value into a state value and action advantages computed by two heads on a shared representation, $Q(s,a) = V(s) + A(s,a) - |\mathcal{A}|^{-1} \sum_{a'} A(s,a')$, where the mean subtraction resolves the identifiability of the decomposition and lets the network learn the value of a state without having to learn the effect of every action in it; this is useful when in many states the choice of action barely matters and in a few it is critical. And an *n-step return* accumulates real rewards over a window of n steps before bootstrapping, $y = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n\, Q_{\theta^-}(s_{t+n}, \cdot)$, trading a little extra variance for much faster credit propagation, a sensible trade when a sparse success signal must travel backwards across long trajectories. In the off-policy setting, the window also introduces a sampling mismatch that is formally uncorrected here, an approximation that works well in practice [21].

Termination vs. truncation. One subtlety becomes central in this project. When a window ends in a *terminal* state the bootstrap term is dropped ($\gamma^n \to 0$): there is no future to estimate. When an episode is merely *truncated* — by an external time limit, say — the future does exist and the agent simply stops observing it, so the correct target still bootstraps from the last state [38]. Conflating the two teaches the network that time-outs are value-zero events, systematically biasing value estimates in time-limited tasks.

2.2 Multi-agent reinforcement learning

When several agents learn and act in the same environment, the MDP generalizes to the *Markov game* (or stochastic game) [29]: a tuple $\langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, P, \{R_i\}, \gamma \rangle$ with agent set $\mathcal{I} = \{1, \ldots, N\}$, where the transition kernel $P(s' \mid s, a_1, \ldots, a_N)$ and each agent's reward $R_i(s, a_1, \ldots, a_N)$ depend

on the *joint* action. Settings are classified by reward structure: *cooperative* (identical or aligned interests), *competitive* (zero-sum), and *mixed* (general-sum). This project is cooperative: every drone shares the same objective, and none has a private one. When, in addition, each agent perceives only part of the state, receiving a private observation o_i correlated with but not determining s, the model becomes a *decentralized POMDP* (Dec-POMDP) [35]. This is the setting of this project, and it is provably intractable to solve exactly [4]: hence the approximate, learning-based methods used in practice, and in this work.

Non-stationarity. The central pathology of multi-agent learning is that, from any single agent's perspective, the environment contains the *other learning agents*: as they update their policies, the transition and reward statistics that agent i experiences drift, violating the stationarity assumption underlying single-agent convergence guarantees [20]. Two classical extremes span the trade-off. *Independent learners* [50] run a single-agent algorithm per agent, ignoring the others: simple and scalable, but the drifting environment can destabilize learning and silently erase coordination. *Joint-action learners* model the joint action space explicitly: coordination-aware, but exponential in N and dependent on observing others' actions.

Centralized training, decentralized execution. Most modern cooperative MARL adopts a middle path, *centralized training with decentralized execution* (CTDE) [27, 31]: during training, typically in simulation, global information may be pooled (shared replay data, joint critics, others' observations), but the artefacts deployed are per-agent policies conditioned only on locally available inputs. CTDE exploits the asymmetry between what is available in the laboratory and what is available in the field, buying training-time stability without sacrificing runtime decentralization.

Parameter sharing. In teams of homogeneous agents, a particularly effective CTDE instance is *parameter sharing* [17]: all agents are driven by a single network, trained on the pooled experience of the whole team. Sharing multiplies the effective sample throughput per parameter, and partially neutralizes non-stationarity at its source, since teammates cannot drift apart when they are, parametrically, the same policy. Its known caveat is symmetry: identically parameterized agents receiving identical observations produce identical behaviour, so useful role differentiation requires an input that distinguishes them, typically an agent index appended to the observation.

Credit assignment. With a shared team reward, each agent faces the question of which part of the outcome its own actions caused; a lazy agent can free-ride on a successful teammate, and naive independent updates propagate the team's success into everyone's values indiscriminately. Practical remedies range from individually shaped reward components, which keep a per-agent learning signal, to explicit value-decomposition schemes; the design tension is that individual shaping can misalign incentives with the team objective if the shaped terms dominate.

Partial observability and memory. Under partial observability the optimal policy generally depends on the observation *history*, not the last observation alone; the principled object to condition on is a belief state [26]. Deep approaches either learn a recurrent summary of the history [19] or hand the agent an explicitly engineered memory maintained outside the network, such as spatial maps accumulating everything observed so far, turning the observation itself into an approximate sufficient statistic of the past. The latter trades representational generality for trainability and transparency, and is the choice taken in this project (Section 3.3).

2.3 Generalization and domain randomization

A policy trained in one fixed configuration tends to overfit it. When the deployment conditions are uncertain, or when one wants a *single* policy to cover a family of related tasks, a standard

Symbol	Default	Meaning
$\mathcal{G}$	$\{0,\dots,31\}^2$	set of grid cells (fixed 32×32 map)
$\mathcal{O} \subset \mathcal{G}$, ρ	$\rho \in [0.10, 0.30]$	obstacle cells and obstacle density
$\mathcal{F} = \mathcal{G} \setminus \mathcal{O}$	—	free (traversable) cells
$g \in \mathcal{F}$	—	hidden target cell
N, i	$N \in \{1,\dots,4\}$	team size and drone index $i \in \{1,\dots,N\}$
p_i^t	—	position of drone i at turn t
$\mathcal{A}$	$\{\text{up}, \text{down}, \text{left}, \text{right}\}$	action set (4 unit moves)
r_{vis}	$r_{\text{vis}} \in \{1,\dots,6\}$	vision radius (Chebyshev)
$V_i^t \subseteq \mathcal{G}$	—	vision set of drone i at turn t
r_{comm}, d_1	$r_{\text{comm}} \in \{2,\dots,12\}$	communication range and Manhattan distance
p_{fault}	$p_{\text{fault}} \in [0, 0.003]$	per-turn fault probability of a live drone
$\beta_i^t \in \{0,1\}$	—	alive flag of drone i (0 = broken, permanent)
K_i^t	—	knowledge state (accumulated maps) of drone i
$\mathcal{C}_i^t$, $\hat{n}_i^t$	—	crashed teammates *known* to i; believed alive count
$T_{\max}$	$200 \cdot N$	episode budget in agent-turns
γ	0.97	discount factor

Table 1: Notation used throughout the report. "Default" reports the value or randomization range used in the final training configuration (Section 5); parameters with a range are resampled at every episode.

remedy is *domain randomization* (DR): sampling the parameters that define the task (in robotics, typically visual appearance or physical dynamics) anew at every training episode, so that the learner experiences a distribution of environments rather than a point [40, 51]. A policy that performs well across the sampled distribution has, by construction, learned strategies robust to the randomized factors; with wide enough ranges, the deployment condition becomes "just another sample".

Randomization can be applied blindly, the policy never knowing which variant it is in and having to behave robustly on average, or with *context conditioning*: the sampled task parameters, or the subset a deployed agent could realistically know, are appended to the policy input, turning the problem into a *contextual* MDP [18] and the network into a task-conditioned family of policies with shared weights. Conditioning lets the same parameters produce different behaviour in different regimes, searching more cautiously with a short sensing range than with a long one, rather than settling on a single compromise behaviour. Which parameters to expose and which to leave latent is a design decision, and one this report settles concretely in Section 4.5: observable, actionable quantities help, while quantities the agent could never know in reality would train a dependence that breaks at deployment.

3 Problem Formulation

This section formalizes the task that the rest of the report builds upon: a team of drones must locate and reach a hidden target on a partially observable grid, under limited communication, a fixed time budget, and the possibility that any drone permanently breaks at any moment. We first describe the scenario informally, then cast it as a partially observable Markov game with sequential execution, and detail its three defining ingredients: the observation model, the communication model and the fault model. The last subsection states the objective and the reward function that operationalizes it, while Table 1 collects the notation used throughout the report.

3.1 Scenario

The environment is an episodic grid world. At the beginning of every episode a fresh 32×32 occupancy grid is generated: each cell is independently declared an obstacle with probability ρ, and a single target cell g is placed uniformly at random among the free cells that are reachable

(under 4-connectivity) from the reference corner $(0, 0)$.[1] A team of N drones spawns *co-located* on one randomly chosen corner of the map. The target position is unknown to the drones; obstacles are likewise unknown until observed.

Each drone occupies one cell and, on its turn, moves to one of the four axis-adjacent cells. Drones perceive only a local neighbourhood of their position, accumulate what they have seen into a private map, and can merge maps with teammates that are close enough. The episode succeeds when any drone *physically reaches* the target cell; merely spotting the target does not end the episode, it only reveals its location to the observer and, via communication, to nearby teammates. The team operates under a hard budget of $T_{\max}$ agent-turns; if the budget expires before the target is reached, the episode fails.

Two features make the problem distinctly harder than classical cooperative exploration. All drones start from the *same* cell, so any efficient division of the search space must be produced by the drones themselves, nothing in the environment assigning regions or roles. And at any turn a live drone may irreversibly break, leaving the survivors to absorb the loss without any central entity telling them a teammate is gone.

3.2 Formal model

We model the task as a partially observable Markov game [29, 35], the multi-agent decision model reviewed in Section 2.2, with identical-interest (cooperative) structure and *sequential execution*,

$$\mathcal{M} \;=\; \langle \mathcal{I},\ \mathcal{S},\ \{\mathcal{A}_i\}_{i\in\mathcal{I}},\ P,\ \{\Omega_i\}_{i\in\mathcal{I}},\ \{r_i\}_{i\in\mathcal{I}},\ \gamma \rangle,$$

where $\mathcal{I} = \{1, \ldots, N\}$ is the set of drones and $\mathcal{A}_i = \mathcal{A}$ for all i.

State. A global state $s^t = (\mathcal{O},\ g,\ (p_i^t)_i,\ (\beta_i^t)_i,\ (K_i^t)_i,\ t)$ collects the episode-fixed obstacle layout and target position, the drone positions and alive flags, each drone's knowledge state and the turn counter. Including the knowledge states is deliberate: the per-drone observation is a deterministic rendering of K_i^t, so the process is Markovian in s^t and partial observability arises precisely from the gap between K_i^t and the full state. Episode-level parameters $\theta = (r_{\text{vis}}, r_{\text{comm}}, N, \rho, p_{\text{fault}})$ are drawn at the start of every episode and held fixed within it, a training-time design choice discussed in Section 5.

Sequential execution. Time advances in *agent-turns*. At turn t the acting drone is $\iota(t) = ((t-1) \bmod N) + 1$, so a *round* consists of N consecutive turns in fixed order. Only the acting drone selects an action; every other drone's position is unchanged. Crucially, a turn is consumed *even when the acting drone is broken*: the clock never skips, so losing drones never stretches the time budget. Sequential execution gives collisions and map updates an unambiguous semantics, there being no simultaneous-move conflicts to resolve, and it matches the execution model of Section 4.

Transition. The transition kernel P composes a stochastic fault event with a deterministic move. At the start of its turn, a live acting drone i breaks with probability p_{fault} (its position freezes, $\beta_i \leftarrow 0$, permanently). Otherwise it executes its action: the move succeeds if the destination cell is inside the grid and not an obstacle, and the drone stays in place if it is not. After moving, the drone's vision, the fault-beacon detection and the communication fusion update the knowledge states. The only sources of stochasticity are the episode initialization (layout, target, spawn corner, θ) and the fault process.

[1] Reachability is verified with a breadth-first search from $(0, 0)$ at generation time; layouts whose reachable region is empty are rejected and resampled. The guarantee therefore concerns $(0, 0)$ alone, and $(0, 0)$ is also the only corner forced free: since the team spawns on a uniformly chosen corner, the target is in fact unreachable from the actual spawn in 1.4% of episodes at $\rho = 0.10$, 8.6% at $\rho = 0.20$ and 69.6% at $\rho = 0.40$. That is a property of the instance distribution, not of the policy; it imposes a density-dependent ceiling on the success rates of Section 7, quantified in Section 7.3.

Action masking. The acting drone receives a binary mask over $\mathcal{A}$ disabling moves that leave the grid or enter an adjacent obstacle. Because $r_{\text{vis}} \geq 1$ the adjacent cells are always inside the drone's vision set, so the mask adds no information beyond what the drone has already observed: it merely removes actions whose outcome (bouncing in place) is known, shrinking the exploration space. A fallback re-enables all actions in the degenerate case of a drone boxed in on all four sides, so the policy always has a legal choice.

Episode end. The episode *terminates* (success) as soon as the acting drone's new position equals g. It is *truncated* (failure) when t reaches $T_{\max} = 200 \cdot N$, or early when all drones are broken, since no state change is possible anymore. Faults are likewise truncations of the broken drone's experience stream rather than terminal states; the learning consequences of this distinction are worked out in Section 5.3.

3.3 Observation model

Each drone senses a square window centred on itself: at turn t, drone i's vision set is the Chebyshev ball

$$V_i^t \;=\; \{\, c \in \mathcal{G} \;:\; \|c - p_i^t\|_\infty \leq r_{\text{vis}} \,\},$$

i.e. a $(2r_{\text{vis}}{+}1) \times (2r_{\text{vis}}{+}1)$ box clipped to the grid. Sensing is noiseless but strictly local: nothing outside V_i^t is perceived.

What the policy consumes is not the instantaneous percept but an *accumulated knowledge state*. Drone i maintains $K_i^t = (\mathit{Vis}_i, \mathit{Obs}_i, \mathit{Tgt}_i, \tau_i, \mathit{Brk}_i, \mathcal{C}_i)$, a set of spatial maps over $\mathcal{G}$ updated after every move: for every cell $c \in V_i^t$,

$$\mathit{Obs}_i(c) \leftarrow 1 \;\text{ if } c \in \mathcal{O}, \qquad \mathit{Vis}_i(c) \leftarrow 1 \;\text{ otherwise, and, in addition,} \qquad \mathit{Tgt}_i(c) \leftarrow 1 \;\text{ if } c = g.$$

All three maps are monotone: knowledge is never forgotten, so a target spotted once stays marked after it leaves the field of view. The trajectory map $\tau_i(c)$ counts how many times drone i has itself occupied cell c and, unlike the others, is *private* (never communicated), acting as a personal revisit memory; the crash-site map Brk_i and the crashed-teammate set $\mathcal{C}_i$ record fault knowledge.

The observation $o_i^t \in \Omega_i$ delivered to the policy is a deterministic, multi-channel rendering of K_i^t plus the drone's own position and the positions of *live* teammates currently inside V_i^t; its tensor encoding is deferred to Section 4.3. Conceptually the accumulated maps are a hand-crafted belief state, compressing the interaction history into a spatial summary, which is what allows a memoryless policy to act reasonably under partial observability.

3.4 Communication model

Communication is proximity-bound, instantaneous and content-fixed: what is exchanged is *world knowledge*, never commands or intentions. After every turn, consider the communication graph over live drones with an edge between i and j whenever their Manhattan distance satisfies $d_1(p_i^t, p_j^t) \leq r_{\text{comm}}$. For every connected component S of this graph, the members merge their knowledge by an element-wise maximum,

$$\mathit{Vis}_i \leftarrow \max_{j \in S} \mathit{Vis}_j, \qquad \mathit{Obs}_i \leftarrow \max_{j \in S} \mathit{Obs}_j, \qquad \mathit{Tgt}_i \leftarrow \max_{j \in S} \mathit{Tgt}_j, \qquad \mathit{Brk}_i \leftarrow \max_{j \in S} \mathit{Brk}_j,$$

together with the union of the crashed-teammate sets $\mathcal{C}_i \leftarrow \bigcup_{j \in S} \mathcal{C}_j$, for every $i \in S$. Merging over connected components makes communication *transitive* at every fusion event: if A is in range of B and B of C, all three end up with identical fused maps even if A and C are far apart, since intermediate drones act as relays.

Three properties of this mechanism carry the multi-agent analysis of Section 6. Fusion is *anonymous*, in that the max-merge keeps no record of who contributed which cell, so shared knowledge detaches from its producer; *persistent*, in that knowledge received in an encounter

survives after the drones part ways, being written into the receiver's own maps; and *selective*, in that the trajectory map τ_i is excluded from fusion, so the revisit-avoidance signal remains individual. Wrecks neither transmit nor receive: a broken drone's maps freeze at fault time. Figure 1 puts vision, communication and a wreck's beacon side by side on one miniature board.

3.5 Fault model

The defining extension of this project is a random, permanent fault process. At the start of its own turn, a live drone breaks with probability p_{fault}, independently across drones and turns. The values used in training are small per-turn probabilities with large episode-level effect: at $p_{\text{fault}} = 0.003$ over 200 rounds, a given drone breaks at some point of the episode with probability $1 - 0.997^{200} \approx 0.45$.

A broken drone (a *wreck*) is subject to the following semantics:

- **Total incapacitation.** It no longer moves, senses or communicates; its knowledge state freezes at fault time.
- **No time refund.** Its turns still occur and still consume the shared budget $T_{\max}$: the team loses capacity but gains no time.
- **Physically inert.** The crash cell is not an obstacle: live drones can traverse it, it never blocks a corridor (which preserves the reachability guarantee of the target), and the wreck is excluded from collision counting and from teammate-position observations.

Decentralized fault knowledge. No drone is notified of a teammate's fault. Instead, a wreck passively emits a *distress beacon*: a live drone i discovers crash site c_j of drone j when it either comes within communication range of it, $d_1(p_i^t, c_j) \le r_{\text{comm}}$, or sees it, $c_j \in V_i^t$. Upon detection, i records the crash in its own maps, $Brk_i(c_j) \leftarrow 1$ and $\mathcal{C}_i \leftarrow \mathcal{C}_i \cup \{j\}$, and this knowledge then spreads through the ordinary map fusion exactly like any other observation. Each drone therefore holds a *belief* about the team's size,

$$\hat{n}_i^t \;=\; N - |\mathcal{C}_i^t|,$$

which is sound but possibly stale: since $\mathcal{C}_i^t$ can only contain drones that actually crashed, $\hat{n}_i^t$ never underestimates the true alive count, but it may overestimate it for many rounds if the crash happened far away. There is no global fault oracle anywhere in the system: every quantity a drone acts upon, including $\hat{n}_i^t$, which is fed to the policy (Section 4.5), is derived from that drone's own observations and from what teammates relayed to it. When the last drone breaks, the episode is truncated early.

3.6 Objective and metrics

The team's objective is to minimize the time at which the target is first reached, or equivalently to maximize the probability of reaching it within the budget and, among successful strategies, to reach it as early as possible. The task is identical-interest: no drone has a private goal, and it is irrelevant *which* drone reaches the target.

This objective is operationalized by the shaped per-turn reward of Table 2. On its turn, a live drone i collects

$$r_i^t \;=\; r_{\text{step}} \;+\; \mathbb{1}[\text{invalid move}]\, r_{\text{wall}} \;+\; \mathbb{1}[\text{valid move}]\,(k\, r_{\text{rev}} + m\, r_{\text{coll}}) \;+\; c\, r_{\text{expl}} \;+\; \mathbb{1}[p_i^t = g]\, R_{\text{tgt}},$$

and the learning objective is the usual expected discounted return with discount γ. Each term implements one facet of the task:

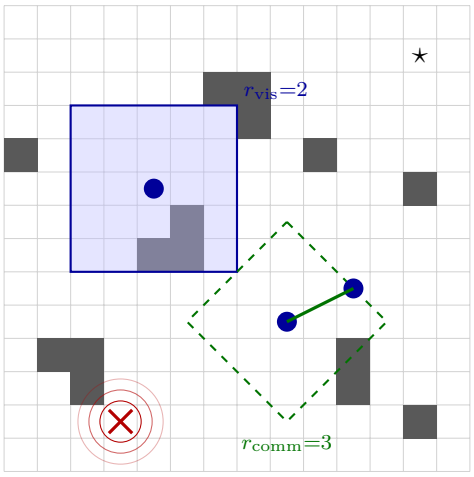

Figure 1: Schematic of the search scenario (illustrative 14×14 crop; the actual map is 32×32). Dark cells are obstacles and $\star$ is the hidden target. The left drone's square outline is its Chebyshev vision window ($r_{\text{vis}} = 2$), inside which cells are revealed and accumulated into its private map. The dashed diamond is the Manhattan communication ball ($r_{\text{comm}} = 3$) of the lower drone: the teammate inside it fuses maps with it on every turn (solid link). The red cross is a wreck; its concentric rings depict the distress beacon, which live drones detect when they come within r_{comm} of the crash site (or see it), with no global notification of the fault.

Term	Symbol	Value	Applied when
step penalty	r_{step}	-0.3	every turn of a live drone (base cost of time)
wall penalty	r_{wall}	-0.02	the move is invalid; the drone stays in place
revisit penalty	$r_{\text{rev}} \cdot k$	$-0.05 \cdot k$	entering a cell drone i already visited k times
collision penalty	$r_{\text{coll}} \cdot m$	$-0.5 \cdot m$	entering a cell occupied by m other live drones
exploration bonus	$r_{\text{expl}} \cdot c$	$+0.02 \cdot c$	c cells newly added to the *team's* discovered set
target reward	R_{tgt}	$+200$	the drone's new cell is the target (terminates)

Table 2: Reward terms accumulated within a single turn of the acting drone (values from the final configuration). A wreck's turns still elapse and still consume the budget, but change nothing and earn no reward.

- *Time pressure and efficient coverage.* The constant step penalty makes every turn costly, so faster acquisition strictly dominates; the exploration bonus counts cells that are new *to the team* (the union of all discovered sets), not to the individual, so re-discovering a region a teammate already swept earns nothing, which rewards complementary rather than duplicated search.
- *Revisit avoidance.* The revisit penalty is linear in the drone's own prior visit count k of the destination cell, so the cumulative cost of oscillating on one spot grows quadratically with the number of passes.
- *Dispersion.* The collision penalty charges co-location with live teammates. Since all drones spawn on the same corner this is the only signal pushing the team to split; no region assignment or coordination protocol is provided, a point developed in Section 6.4.
- *Team credit.* With the cooperative variant used in the final configuration (`shared_target_reward`), R_{tgt} is granted to the whole *surviving* team rather than only to the finder, aligning individual returns with the identical-interest objective; the learning-side implications are discussed in Section 5.3.

At evaluation time the reward is set aside and performance is reported through task-level metrics: *success rate* (fraction of episodes in which the target is reached within $T_{\max}$), *time-to-find* (rounds elapsed at termination, i.e. agent-turns divided by N, over successful episodes), *coverage* (fraction of free cells discovered by the team), and *redundancy* (overlap between the regions swept

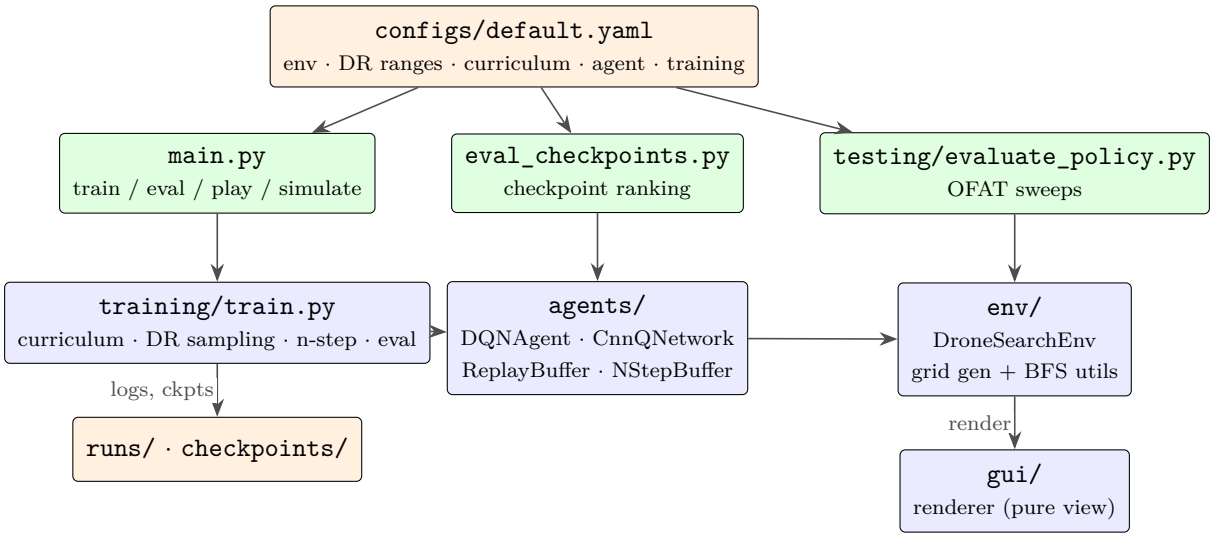

Figure 2: Module layout and main data paths. Every entry point reads the same YAML configuration; the training loop drives the shared agent and the environment; the GUI is a pure view, never a controller. Import dependencies form a DAG (no cycles): the network depends only on the deep learning framework, the agent on the network and buffers, the environment on its own utilities.

by different drones). The last two also serve as quantitative signatures of the emergent division of labour analysed in Section 6.6.

4 System Architecture

The system is implemented from scratch in Python. This section walks through its architecture bottom-up: the codebase layout and its configuration-driven design, the environment implementation, the tensor encoding of the observations of Section 3.3, the neural network that maps observations to action values, the context vector that conditions it, the learning machinery around it, and the interactive tooling used for inspection and evaluation.

4.1 Overview and module layout

The codebase separates four concerns (Figure 2): the *environment* (`env/`) implements the world model of Section 3; the *agent* (`agents/`) holds the network, the replay and n-step buffers and the update rule; the *training package* (`training/`) owns the episode loop, curriculum staging and domain-randomization sampling; and the *GUI* (`gui/`) renders episodes without ever influencing them. The entry points are thin wrappers over these packages.

A single YAML file is the sole source of truth for every tunable quantity: geometry and reward values, randomization ranges, curriculum phases, network and optimizer hyperparameters, logging cadence. The consequence is methodological: an experiment, including the full training curriculum, is specified by editing configuration, not code. The same discipline governs derived quantities; the normalizers of the context vector, for instance, are computed from the randomization ranges rather than stated independently, so widening a range cannot silently desynchronize the network's inputs.

4.2 Environment implementation

`DroneSearchEnv` realizes the model of Section 3.2 behind a Gymnasium-style interface [52], with one essential extension: the native stepping primitive is `step_agent(i, action)`, one call per drone per round in fixed round-robin order, rather than a joint step (a standard `step(action)` wrapper exists for API compliance, but the training loop drives `step_agent` directly). Episode setup follows a strict contract: `set_domain_params(...)`, which installs

Ch.	Name	Stream	Encoding
0	Visited	both	$\{0,1\}$: free cells discovered by drone i (incl. fused)
1	Obstacle	both	$\{0,1\}$: obstacles discovered by drone i
2	Trajectory	both	$\min(\tau_i(c)/5,\, 1)$: own visit counts, saturating
3	Target	both	$\{0,1\}$: target cell once seen (persistent memory)
4	Own_Position	global	$\{0,1\}$: single 1 at drone i's cell
5	Broken	global	$\{0,1\}$: crash sites *known to drone* i
4	Other_Position	local	$\{0,1\}$: *live* teammates currently within V_i^t

Table 3: Observation channels. The *global* stream ($6 \times 32 \times 32$) is consumed whole; the *local* stream ($5 \times 32 \times 32$) is cropped to a 13×13 patch centred on the drone before encoding. Channel 4 differs by stream: the egocentric patch needs no own-position marker, but benefits from seeing nearby teammates. Wrecks never appear as teammates: they surface in the Broken channel instead, once known.

$\theta = (r_{\text{vis}}, r_{\text{comm}}, N, \rho, p_{\text{fault}})$, then `reset()`, which generates the world. Keeping the two separate matters because the same parameters are also fed to the network as its context vector: the contract guarantees that what the environment *does* and what the network is *told* can never diverge.

World generation. `reset()` performs rejection sampling: it draws an obstacle map with density ρ, forces the reference corner free, computes the BFS-reachable region, and resamples (up to a bounded number of attempts) if that region is trivial; the target is then drawn uniformly from the reachable free cells. All randomness flows through a single seeded generator, and map generation consumes it before the fault process does, so one seed reproduces the layout *and* the fault sequence, a property the evaluation protocol relies on.

Anatomy of a turn. A live drone's `step_agent` call executes a fixed pipeline: (i) the fault coin flip (on a break, the rest of the pipeline is skipped and the call returns a flagged no-op, the turn having been spent all the same); (ii) move validation and execution, accumulating the reward terms of Table 2; (iii) vision update of the drone's knowledge maps; (iv) wreck-beacon detection; (v) communication fusion over the connected components; (vi) update of the team-level discovered set, from which the exploration bonus is computed; (vii) target check. A wreck's call skips everything but the clock. The action mask is computed on demand per drone and returned alongside the observation, so every consumer (training, evaluation, GUI) applies exactly the same legality rule.

4.3 Observation encoding

The knowledge state K_i^t reaches the network as two spatial tensors (Table 3), both full-map, normalized to $[0, 1]$:

Two encoding decisions deserve justification. First, **the drone's own position is a spatial channel**, not an out-of-band coordinate pair: channel 4 of the global stream holds a single 1 at the drone's cell, so the convolutional encoder can reason about position *spatially*, in its relation to obstacles, to the searched region and to known crash sites, while still recovering the integer coordinates exactly (arg max over that channel) whenever it needs them, as the patch-cropping step does. Second, **fault knowledge is spatial too**: the Broken channel marks the crash sites drone i knows about, and only those. Beyond the scalar belief $\hat{n}_i$, the *location* of a crash tells the network *which region* lost its searcher, precisely what is needed to re-cover a dead teammate's sector. The local stream then complements the global one at a different scale: its 13×13 crop, sized to match the largest vision window ($2 \cdot 6 + 1 = 13$), retains full resolution where cell-level distinctions drive immediate action.

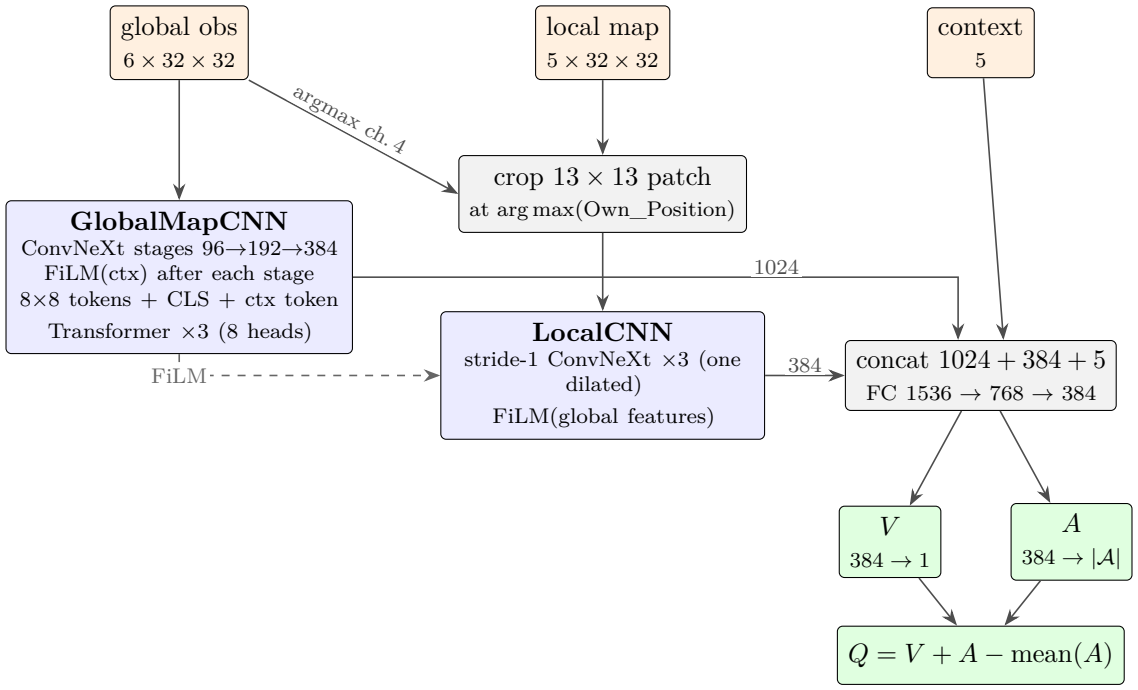

Figure 3: The shared Q-network. Solid arrows carry activations; dashed arrows carry conditioning (FiLM modulation and the context token). The same network, with the same weights, drives every drone; per-drone behaviour arises only from per-drone inputs.

4.4 Neural architecture

A single network (Figure 3) maps $(o_i^{\text{glob}}, o_i^{\text{loc}}, \mathbf{c}_i)$, that is global stream, local stream and context vector, to the four action values. It fuses three information pathways: a *global encoder* for strategic structure, a *local encoder* for tactical detail, and the context for regime awareness.

Global encoder. The global stream is processed by a ConvNeXt-style hierarchy [30]: three stages of inverted-bottleneck blocks (depthwise 7×7 convolution, pointwise expansion, GELU, and Global Response Normalization [59]) with channel widths $96 \to 192 \to 384$ at spatial resolutions $32 \to 16 \to 8$, regularized by stochastic depth [22]. After each stage a FiLM layer [41] applies a context-conditioned affine transformation $\text{FiLM}(x) = \gamma(\mathbf{c}) \odot x + \beta(\mathbf{c})$ (initialized to identity), letting the sampled regime modulate feature extraction at every scale. The final 8×8 map is flattened into 64 tokens, augmented with a learned CLS token and a projected context token, and passed through a 3-layer Transformer encoder [55], the CLS output being concatenated with the mean of the spatial tokens and projected to a 1024-dimensional embedding. This convolution-plus-attention hybrid, in the spirit of bottleneck transformers [47], is deliberate: a plain flatten-or-pool head would collapse the spatial layout precisely where the task needs *relational* reasoning over distant regions ("the unexplored area lies beyond that wall, on the far side of the known crash site").

Local encoder. The 13×13 patch, cropped at the coordinates recovered from the Own_Position channel, is encoded by three ConvNeXt blocks at constant width 96 with *no* spatial downsampling (one block dilated to widen the receptive field): on a grid where every cell matters, a stride would destroy exactly the cell-level distinctions the patch exists to preserve. A FiLM layer conditioned on the *global embedding* rather than on the raw context then modulates the local features, so that the strategic picture reshapes how tactical surroundings are read, before projection to a 384-dimensional embedding.

Dueling head. Global embedding, local embedding and raw context are concatenated ($1024 + 384 + 5 = 1413$) and passed through a three-layer fully connected trunk ($1536 \to 768 \to 384$, each with layer normalization and GELU) that feeds the dueling decomposition of Section 2.1: separate value and advantage heads recombined as $Q = V + A - \text{mean}(A)$.

Normalization discipline. The network uses layer normalization [3] and GRN exclusively, no batch normalization anywhere, for an operational reason: batch statistics would make the batch-of-one greedy inference each drone executes at every turn behave differently from a batched training forward pass. With batch-size-independent normalization the two are identical by construction; the only module that distinguishes them is stochastic depth, inactive at inference.

4.5 Context vector

The context $\mathbf{c}_i \in [0,1]^5$ is the interface between domain randomization and the network:

$$\mathbf{c}_i \;=\; \Big(\tfrac{r_{\text{vis}}}{r_{\text{vis}}^{\max}},\; \tfrac{r_{\text{comm}}}{r_{\text{comm}}^{\max}},\; \tfrac{N}{N^{\max}},\; \tfrac{i}{N^{\max}},\; \tfrac{\hat{n}_i}{N^{\max}}\Big),$$

where the normalizers are the *maxima of the randomization ranges*, derived programmatically from the configuration. Each component has a distinct role:

- *Sensing and communication ranges* tell the policy which regime it is operating in, enabling regime-dependent behaviour: how much a corridor can be trusted as "explored" given the width of the vision window, for instance.
- *Team size and agent identity.* The identity term is the symmetry breaker required by parameter sharing (Section 2.2), and here it is not optional: all drones spawn on the *same* cell with identical knowledge, so without i in the input a shared deterministic policy would move them identically, forever. With it, the network can learn an implicit role assignment: conditioned on identity, disperse in different directions.
- *Believed alive count* $\hat{n}_i$ is the fault model's entry point into decision-making. As the believed team shrinks, the division of labour that made sense for N drones no longer does, and the policy can recalibrate, widening its own effective sector.

Because $\hat{n}_i^t$ evolves within the episode, the context is *rebuilt at every turn for every drone*; it is a per-decision input, not an episode constant. Deliberately *absent* from the context are the two remaining randomized parameters: obstacle density, which is redundant since the network sees actual obstacles in its maps, a far richer signal than a scalar summary; and fault probability, which is unknowable, since a real drone has no access to its own failure rate, so training a dependence on it would build in an input that deployment cannot supply, exactly the pitfall noted in Section 2.3.

4.6 Learning machinery

Experience pipeline. Every drone's transitions flow through a per-drone n-step window into one *shared* replay buffer (uniform sampling, 2×10^5 capacity): parameter sharing extends to experience, and the network trains on the pooled stream of the whole team. Per-drone windows are necessary because execution is sequential: drone i's "next step" is one round away, interleaved with every other drone's turns, so windows must be threaded by agent rather than by wall-clock order. Each emitted transition carries the pair $(R^{(n)}, \gamma^n)$, the discounted n-step return and the bootstrap discount, with γ^n replaced by 0 if the window hit a terminal state; so termination-vs-truncation semantics are decided *once*, at emission time, and the update rule stays oblivious to them. The window also implements the two fault-sensitive behaviours motivated in Section 5.3: an immediate bootstrapped flush when its drone breaks, and retroactive team crediting of the terminal bonus, which requires the terminal round to still be pending (hence $n \geq 2$).

Update rule. Training is standard Double-DQN regression on the target of Section 2.1, computed from the stored $(R^{(m)}, \gamma^m)$ pair, minimizing the Huber loss under gradient-norm clipping with hard target synchronization; one gradient step runs every few agent-turns, so learning interleaves with acting at a fixed ratio. Hyperparameter values are consolidated in Appendix A.

Batched decentralized acting. At every round, action selection for the whole team costs at most one forward pass: each live drone independently draws its exploration coin (ε-greedy), the greedy drones' inputs are stacked into one batch, Q-values are computed jointly, and each drone's action mask is applied to its own row before the arg max (masked actions set to $-\infty$; exploring drones sample uniformly among their legal actions). This is an implementation convenience enabled by parameter sharing: the *information* used for each drone's decision remains strictly its own observation and context, preserving decentralized-execution semantics. One consequence is worth stating: since the batch is formed at the round boundary and then applied turn by turn, a drone acts on its observation as of its *previous* turn, so it does not see the cells a teammate uncovered earlier in the same round. The staleness is bounded by one round and never crosses drone boundaries; it is, if anything, a conservative model of a real team, whose members would not have instantaneous access to each other's latest percept either.

4.7 Tooling and interaction modes

Beyond training, the same environment and agent serve three interactive workflows: an *eval* mode that renders greedy policy episodes; a *play* mode that hands the first drone to the keyboard, valuable for sanity-checking reward mechanics and fault behaviour from the inside; and a *simulate* mode with a configuration screen (team size, vision, communication, density, and a fault-probability slider) that runs batches of episodes with live rendering of maps, beliefs and per-drone status (Figure 4). The renderer is strictly read-only with respect to the environment (Figure 2).

For quantitative work, `eval_checkpoints.py` ranks checkpoints on identical fixed-seed instance sets, the same seeds reproducing the same layouts *and* fault sequences, while `testing/evaluate_policy.py` runs the one-factor-at-a-time (OFAT) sweeps of Section 7. Both support an optional *auto-navigation* override: once a drone's maps contain the target, a BFS planner on its *known* map (unknown cells optimistically assumed free, replanned every step) steers it there deterministically. The override exists to decouple two abilities in analysis, *finding* the target versus *reaching* a found target, and is confined to evaluation tooling by an architectural invariant: it never runs during training, nor during the in-training evaluation that selects checkpoints. In the training loop it would corrupt the data distribution, since transitions would reflect the planner's policy rather than the network's; in checkpoint selection it would corrupt the metric, checkpoints being ranked partly on the planner's competence.

5 Training Methodology

This section describes how the shared policy is trained: the domain-randomization protocol, the exploration and learning-rate schedules, the learning-side decisions that the fault model makes unusually delicate, and the infrastructure that makes a long run reproducible and resumable.

5.1 Domain randomization

The training loop supports staging a run as an ordered list of *phases*, each with its own episode count and optional overrides; a phase with zero episodes is skipped, which makes alternative stagings cheap to keep in the configuration file and to enable one number at a time. The reported policy makes no use of that machinery. It is trained from scratch in a *single* phase of 50,000

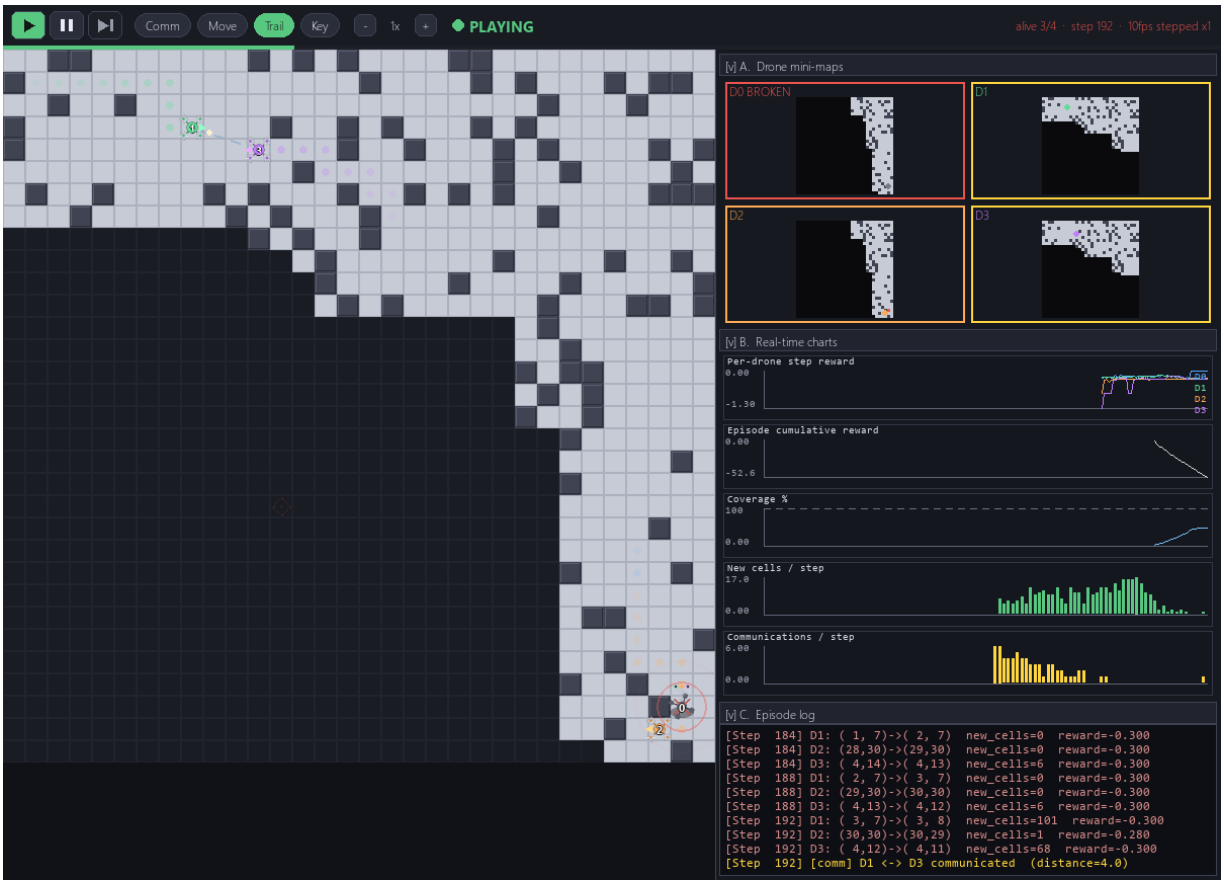

Figure 4: The *simulate* mode mid-episode, with the trained policy driving four drones under an active fault process (round 48, three drones alive). On the board, light cells are the team's discovered region and dark ones the unknown remainder; the dashed link at the top left is an active fusion event between two drones inside r_{comm}, and the red concentric rings at the bottom right are the distress beacon of the wreck, which the drone beside it has just entered range of. The sidebar is the instrumentation the analysis relies on: the four mini-maps show each drone's *private* accumulated knowledge, visibly different from one another and from the team union, with the wreck's frozen map flagged `BROKEN`; the charts track per-drone reward, coverage, newly discovered cells and communication events; and the log records every turn, including the fusion events by which crash knowledge spreads.

episodes, with the per-drone budget set to 200 rounds and the full ranges of Table 1 active throughout, faults included (Table 8). Earlier exploratory stagings informed the hyperparameters but contribute no weights.

At the start of every episode the loop samples $\theta = (r_{\text{vis}}, r_{\text{comm}}, N, \rho, p_{\text{fault}})$, integers and floats uniformly from their ranges, applies it to the environment via `set_domain_params`, and only then resets. The ranges are wide by design. Vision spans myopic (r_{vis}=1: a 3×3 window) to far-sighted (r_{vis}=6); communication spans nearly-isolated to easily-connected teams; team size spans a single drone, where cooperation is moot and the policy must search alone, up to four; obstacle density spans open fields to cluttered mazes; and the fault range spans "faults never happen" to "losing roughly half the team over an episode is ordinary". The policy is therefore forced to be simultaneously a competent solo searcher, a cooperative team member and a survivor of team collapse, with the context vector telling it, at every turn, which of these regimes it currently inhabits. Per-episode resampling also acts as an implicit curriculum in expectation: easy episodes (large vision, no faults, sparse obstacles) and hard ones (blind, faulty, cluttered) interleave freely, which keeps a steady stream of successful trajectories, and hence of terminal reward signal, flowing into the replay buffer even before the policy masters the hard regimes.

5.2 Schedules

Exploration. ε-greedy exploration is annealed *per episode* and multiplicatively: from $\varepsilon_0 = 1$, each episode applies $\varepsilon \leftarrow \max(0.05,\, 0.9995\,\varepsilon)$, so the floor of 0.05 is reached at episode 5,990, about 12% of the run (Figure 15a, Appendix C); the residual 5% of random actions thereafter keeps the buffer from collapsing onto the greedy policy's own trajectory distribution. Exploration coins are drawn per drone per turn, so within one round some drones may explore while the others act greedily.

Learning rate. The learning rate follows a linear warmup (10% of the run, $10^{-4} \rightarrow 5 \times 10^{-4}$) into a cosine annealing down to 10^{-6} (Figure 15b). It is stepped *once per episode* rather than per gradient step, because episode lengths vary wildly across randomized regimes: a lucky find ends an episode in tens of turns, a blind faulty maze consumes the full budget, so stepping per update would tie the schedule's shape to the difficulty mix rather than to training progress. And it is sized by the *total* episode count across all phases, so the decay is a property of the run, not of any single phase.

Update cadence. One Double-DQN gradient step runs every 4 agent-turns on minibatches of 16 transitions from the shared replay buffer, with the target network hard-synchronized every 500 gradient steps. Appendix A consolidates all values.

5.3 Learning with an exogenous fault process

Applying the termination/truncation distinction of Section 2.1 to an exogenous fault process gives the decision that matters here: **a fault is a truncation, not a terminal**. When a drone breaks, its pending n-step window is flushed immediately *with* bootstrapping and no further transitions are generated for it. A fault *feels* terminal for that drone, but p_{fault} is exogenous: it strikes independently of state and action, so the states that happened to precede a fault are not worth less than identical states that did not. Zeroing the bootstrap would charge the value function for bad luck, depressing the values of whatever the drone was doing when it broke, with the bias growing in p_{fault}.

Three decisions follow the same discipline. Running out of budget is likewise a truncation, so the final window still bootstraps [38]. The shared terminal bonus of Table 2 is written retroactively into each other live drone's most recent still-pending window, which requires $n \geq 2$ and excludes drones that broke before the find, their windows having already been flushed: a drone that died exploring an empty quadrant should not learn that dying there is worth the team's eventual success. And checkpoint selection is shielded from the shaped reward: every 1,000 episodes the policy is evaluated near-greedily ($\varepsilon = 0.05$) on a *fixed* bank of 20 seeded instances, and a new best requires a strictly higher success rate, mean reward breaking ties only at 100% success, so a policy is never preferred for harvesting exploration bonuses while finding the target less often.

5.4 Infrastructure and reproducibility

Training runs on a SLURM-managed HPC cluster (one GPU per run); the identical entry point runs unchanged on a workstation, which is how short debugging stagings were iterated. Because the cluster's per-job wall-clock limit is shorter than a 50,000-episode run, the final run was executed as a *sequence* of jobs, each resuming from `last.pt`, which carries optimizer, schedule position, ε, gradient-step counter and curriculum position, and therefore continuing *as if uninterrupted*. Every run logs per-episode scalars to TensorBoard (reward, length, success, the sampled randomization values, drones lost to faults, ε and the learning rate), so a run's difficulty mix and its learning progress can be inspected jointly; and one seed reproduces the layout *and* the fault sequence, which is what makes checkpoints and configurations comparable on identical instances. Each job writes its own event file, so Figure 5 reports the first tranche, the 30,000 episodes that fitted

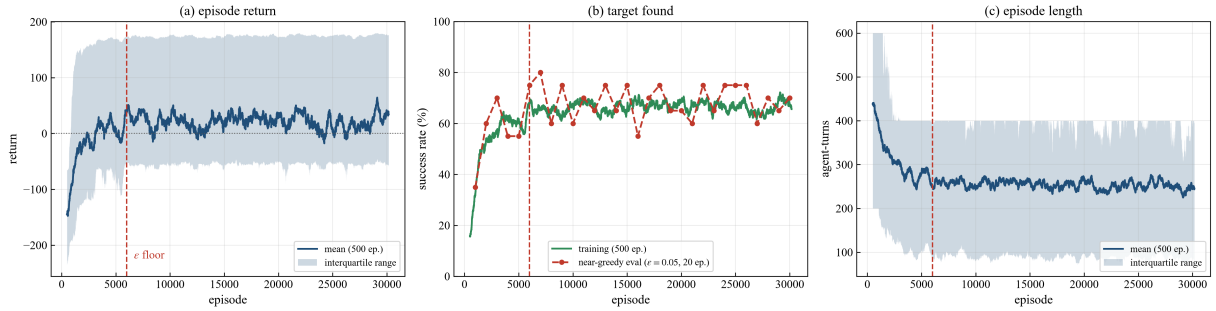

Figure 5: Per-episode training telemetry over the first 30,000 episodes of the final run. Solid curves are 500-episode moving averages, shaded bands the corresponding interquartile ranges; the dashed line marks the episode at which ε reaches its floor. *(a)* Mean return crosses zero at around episode 2,500 and settles near +25; *(b)* the target is found in 65–68% of episodes at the plateau, with the near-greedy evaluation (red, $\varepsilon = 0.05$) tracking the exploratory training curve; *(c)* mean episode length falls from 440 to roughly 250 agent-turns, that is, from 171 to 101 rounds once each episode is divided by its own sampled team size. The bands stay wide because every episode draws a fresh randomization θ: their width measures the difficulty mix, not instability.

inside a single allocation. It is the informative one: every logged quantity has flattened by roughly episode 7,000 and none trends over the 23,000 episodes that follow, and the checkpoint-level learning curve of Figure 9 (Section 7.3), reconstructed by re-evaluating the saved weights, covers the full 50,000 episodes anyway.

6 The System Through the Multi-Agent Paradigm

The preceding sections described *what* was built and *how* it is trained. This section changes register and asks what kind of system it is. Multi-agent systems are not merely a technology but an engineering perspective: a way of decomposing a problem into autonomous, interacting, situated components and of locating where a system's intelligence actually resides [24, 60]. Read through that lens the drone team instantiates a remarkably wide slice of the multi-agent conceptual apparatus, from agenthood and the environment-as-abstraction through coordination, artefacts, self-organization, resilience and graduated autonomy to multi-agent learning. Throughout, every conceptual claim is anchored to a concrete mechanism from Sections 3–5, and the section closes by asking, honestly, what a richer multi-agent design would add and at what cost.

6.1 Why a multi-agent formulation

It is worth beginning with the question the rest of the section presupposes: why is this a multi-agent system at all, rather than one program steering four vehicles? The answer is that the defining constraints of the task, not implementation taste, rule the centralized alternative out along the three dimensions that classically motivate the agent paradigm: distribution, autonomy and interaction [24, 62].

Distribution of sensing and knowledge. No component of the system ever holds the global state. Each drone perceives a Chebyshev window of radius r_{vis} and knows, beyond that, only what it has accumulated or been told. A central controller would need the union of all observations at every turn, but the communication model is range-limited: drones farther apart than r_{comm} cannot exchange anything. Centralized control over a communication-limited team is not merely inefficient; during the frequent, deliberate intervals in which the team is partitioned into disconnected communication components, it is *physically unimplementable*. The problem itself is distributed, so the control must be.

Autonomy as a fault-tolerance requirement. The fault model sharpens the argument to its most concrete form. A centralized controller is a single point of failure, and this project's central premise is that components *do* fail, at random, mid-mission. If the deciding entity resided in one drone and that drone broke, the team would be decapitated. In the actual design, every drone

carries the full decision-making apparatus, its own copy of the shared policy, and the death of any subset of drones leaves every survivor as capable as before. Autonomy here is not an aesthetic commitment: it is what makes graceful degradation possible at all.

Interaction as the source of system-level competence. Finally, the task is one no single drone solves well: a lone searcher covers area linearly in time, while a well-dispersed team covers it in parallel. The system-level competence, sweeping the map quickly, avoiding duplicated effort and re-covering the sectors of fallen teammates, is located in no individual policy evaluation; it arises *between* the drones, from their interactions through fusion, mutual observation and shared spatial traces. That is precisely what distinguishes a multi-agent system from a parallel implementation of a single-agent one [60].

One clarification prevents a recurring misunderstanding: the drones share network *weights*, and weight sharing is a training-time device, not a runtime coupling. At execution each drone evaluates its own copy of the function on its own inputs; no activation, gradient or decision crosses drone boundaries. Four copies of the same brain, fed four different experiences, are four agents, just as identical twins are two people.

6.2 Drones as agents

"Agent" is a contested term, but a broadly shared core defines it as a computational system situated in an environment and capable of autonomous action in pursuit of its design objectives, with the *weak notion of agency* requiring autonomy, reactivity, proactiveness and social ability [12, 61]. This subsection audits the drone against each requirement, and is equally explicit about which stronger notions the drone does *not* satisfy.

Autonomy: encapsulated control. The most fundamental computational reading of autonomy is encapsulation of control: an agent is a locus of decision that cannot be *invoked*, other components being able to influence it only through data, never by calling its behaviour as one calls a procedure [61]. The drones satisfy this literally. Nothing in the system commands a drone: teammates cannot move it, interrupt it or query it, and the only thing that ever crosses the boundary between two drones is world knowledge, through fusion. Each turn consists of reading *its own* observation and context, evaluating *its own* policy copy, and acting. Even the round-robin scheduling does not breach encapsulation: it serializes *when* drones act, a property of the environment's clock, not *what* they decide.

Situatedness. Agents act *in* and *through* an environment, and their action cannot be divorced from the context in which it occurs [48]. The drone's entire interface to the world is environmental: it perceives by revealing cells around its position, acts by moving to adjacent cells, and even its communication is situated, fusion being triggered by spatial proximity rather than by addressing a recipient. There is no action-at-a-distance anywhere in the system, and the policy's inputs are spatial through and through: position, fault knowledge and history all reach the network as *maps*, an architectural echo of the fact that for this agent context is location.

Reactivity, with state. The classical formalization of a non-trivially reactive agent equips it with internal state: a perception function from environment states to percepts, a state-update function folding each percept into an internal state, and an action function from internal states to actions [60]. The drone realizes the triple exactly: perception is the vision update (cells within V_i^t, plus beacon detection); the state update is the monotone accumulation into K_i^t, merged with teammates' contributions by fusion; the action function is the Q-network's masked greedy selection over the rendering of K_i^t. That internal state is what lifts the drone above a thermostat-style pure reflex: two drones on the same cell with the same vision act differently if their *histories* differ, because their maps do.

Proactiveness: distilled, not represented. Agents are expected to exhibit goal-directed behaviour, taking initiative rather than merely responding to stimuli [61]. The drone is unambiguously goal-directed in behaviour, since it systematically expands coverage, avoids re-searching and homes in on the target once its location enters the map, yet nowhere in the drone does the goal *exist as an object*. There is no symbol denoting "find the target", no explicit desire, no plan. The goal-directedness was distilled at training time from the reward function of Table 2 into the value estimates that now drive action selection. In the classical dichotomy, the drone is *goal-oriented* rather than *goal-governed*: its objective is implicit in the structure of its behaviour rather than explicitly represented and reasoned over. This is a genuine trade-off, not a defect, and Section 6.10 discusses what explicit goals would buy, and cost.

Social ability. The drone's social repertoire is narrow but real: it donates its accumulated knowledge to whoever comes within r_{comm}, incorporates teammates' knowledge symmetrically, perceives nearby live teammates through the Other_Position channel, and both emits, passively as a wreck, and consumes distress information. The next subsection analyses why this thin, language-free interaction layer is nonetheless a coordination mechanism of some sophistication.

What the drone is not. Against the *strong* notion of agency, with mental states, beliefs and intentions in the technical sense of belief–desire–intention architectures [6, 43], the drone plainly falls short, and it is worth being precise about where. The per-drone knowledge state is belief-*like*: private, partial, possibly stale, updated by observation and testimony. But it is a fixed-schema spatial data structure, not a revisable base of propositions; the drone cannot represent "teammate 2 probably went north", let alone reason about what teammate 2 believes. Deliberation, in the sense of weighing candidate intentions, is likewise absent, replaced by a single amortized function evaluation. The system thus occupies a definite point in the agent design space: fully agentive in the weak sense, deliberately sub-cognitive in the strong one.

6.3 The environment as a first-class abstraction

A recurring lesson of multi-agent engineering is that the environment is not scaffolding around the agents but a design dimension in its own right: an active entity with its own state, dynamics and rules, through which agents perceive, act and interact [58]. This project embodies that stance structurally: `DroneSearchEnv` is not a thin arena but the locus of most of the system's semantics. The agent package is larger in absolute size, but almost all of it is network definition and learning machinery; the world model, including every rule of interaction, lives entirely on the environment side. Four environmental responsibilities deserve explicit notice. *Topology and locality*: the grid, its Chebyshev vision neighbourhoods and its Manhattan communication distances define what "near" means, and thereby delimit perception, interaction and fusion; locality is not a property agents happen to have but a rule the environment enforces uniformly on everyone. *Dynamics*: the fault process lives in the environment, not in the agents, since drones do not decide to fail, the world fails them, and the same holds for the episode clock, including the rule that wrecks' turns still consume budget. *Mediation*: every interaction between drones, whether fusion, mutual sighting or beacon detection, is computed by the environment as a side effect of spatial configuration, so drones have no channel to each other that bypasses the world, which is exactly the situated-interaction discipline Weyns et al. [58] argue for. And *governance*: the action mask constrains the space of legal behaviour at its source, while the reward terms of Table 2 are environmental legislation, the incentive structure under which autonomous behaviour is shaped without ever being commanded.

The methodological payoff appeared already in Section 4: because the environment owns the rules, the agents could be swapped (random, keyboard-driven, BFS-guided, learned) with zero change to the world's semantics, which is what lets Section 7 run the learned policy and the BFS auto-navigation override over identical seeded instances.

6.4 Interaction and coordination

Coordination, in the sense of managing the interdependencies between concurrent activities so that a collection of agents behaves as a coherent ensemble, is the central problem any multi-agent system must solve. This system solves it with a combination that is instructive precisely because of what it lacks: there are no messages, no protocols, no negotiation, no roles assigned by any authority. Coordination arises from three mutually reinforcing mechanisms.

(a) Knowledge-level communication without a language. When two drones fuse maps, what travels is world knowledge ("these cells are free, these are obstacles, the target is here, teammate 3 crashed there") and nothing else: no requests, commands, commitments, or any of the performatives that agent communication languages provide [11]. The omission is principled rather than lazy. Speech acts exist to coordinate *deliberating* agents: a request makes sense when the receiver can decide whether to honour it. Between sub-cognitive policies there is no deliberation to address, and the task is identical-interest, so there is nothing to negotiate over. What remains genuinely useful to exchange is exactly what *is* exchanged: factual knowledge, whose fusion is monotone (element-wise maximum) and therefore order-independent, idempotent and conflict-free. The design question "what should agents say to each other?" is answered here with "nothing; they should compare notes".

(b) The fused map as a shared dataspace. The resulting pattern has the character of a *blackboard* or generative dataspace rather than of message passing [13, 34]: the three properties of generative communication, namely that emitted information outlives its producer, is accessible by content rather than by address, and decouples interacting parties in time and identity, all hold of the fused knowledge. A cell discovered by drone 1 at round 10 shapes drone 4's decision at round 150, long after the encounter that transmitted it, possibly via a relay chain (1 fused with 2, later 2 with 4) such that 1 and 4 were *never* within range of each other; and because fusion is an anonymous maximum, knowledge carries no provenance, the receiver acting on "the target is at (r, c)" identically whether it saw the target itself or inherited the fact third-hand. The team's effective medium of coordination is thus a distributed, eventually-consistent shared map, differing from a classical blackboard in having no central board: every drone carries its own replica, and proximity events are the synchronization protocol.

(c) Stigmergic traces, virtualized. The third mechanism is coordination through the *traces* of activity, the pattern Grassé named stigmergy: work performed modifies the environment, and the modified environment directs subsequent work [15, 39]. The Visited and Trajectory channels are precisely such traces. A drone's movement writes a growing mark into the shared knowledge (visited cells, own visit counts); those marks then *repel* future search, economically via the revisit penalty and the nullified exploration bonus, behaviourally via whatever avoidance the trained policy has internalized, steering drones away from consumed regions exactly as an evaporating pheromone steers ants away from depleted paths, with the sign inverted: repulsion, not attraction. One twist distinguishes this from textbook stigmergy. The traces are not written into the physical grid, the world's cells having no memory of visits, but into the drones' *shared beliefs*, propagating through fusion rather than sitting at a location. It is stigmergy over a distributed cognitive medium rather than over matter; the coordination logic (work marks the field; marks direct work) is identical.

Learned conventions instead of designed protocols. What binds these mechanisms into actual coherence, namely *who* goes where and *how* the team splits, is not specified anywhere. It is a convention learned by co-training. During training, all drones' experience flows into one network; behaviours that reduce duplicated coverage were systematically reinforced by the team's shared exposure to the collision penalty, the team-novelty exploration bonus and the shared

terminal reward. The identity input of Section 4.5 gives the convention something to condition on: from the very first round, when all drones stand on the same cell with identical maps, the policy can map *different identities to different headings*, realizing an implicit division of the map without any sector ever being represented, assigned or agreed upon. This is coordination knowledge stored in weights rather than in protocol state, with consequences, adaptivity and opacity in equal measure, that Sections 6.10 and 8.5 examine.

6.5 An artefact perspective

The agents-and-artefacts view of multi-agent systems holds that a MAS is populated not only by agents but by *artefacts*: non-autonomous, function-oriented computational entities that agents use, which mediate their activities, and in which designers can embed coordination machinery [37]. Artefacts do not pursue goals; they provide operations, behave predictably, and serve. The distinction is illuminating here because it cleanly separates the system's two kinds of components, and locates several load-bearing pieces of the design on the artefact side of the line.

The clearest instance is the *accumulated map*: a cognitive artefact with a definite function (aggregate spatial knowledge), a usage interface (written by moving and sensing, read at every decision as the network's input) and entirely reactive, predictable dynamics (monotone accumulation and max-merge; it never acts on its own). It mediates agent–environment interaction, since the drone acts on the world *as represented by its map*, and through fusion agent–agent interaction as well: the fused map is a coordination artefact in exactly the sense in which tuple spaces and blackboards are [13, 37]. The same reading covers the *distress beacon*, since a wreck is, in death, reduced to an artefact that manifests one fact to whoever passes, and the *action mask*, a constraining artefact that does not decide for the agent but delimits its option space, artefacts enabling and constraining in the same gesture.

Two observations sharpen the picture. Artefacts are where the *designer's* intelligence lives at runtime: the policy is learned and opaque, while the map semantics, the fusion rule, the mask and the reward machinery are designed and transparent; and the system's verifiable properties (fusion never loses knowledge, masks never permit illegal moves, wrecks never block corridors) are all artefact properties, inspectable and predictable by construction [37]. And the tooling of Section 4.7 extends the taxonomy to the humans in the loop: the GUI, the TensorBoard scalars and the seeded evaluation harness are observation artefacts through which the *engineer* perceives the MAS, the instrumentation layer that makes an otherwise opaque adaptive system engineerable at all.

6.6 Self-organization and emergence

Self-organization denotes the acquisition of structure by a system through internal, distributed processes, without external control [2, 7]; emergence denotes the appearance of coherent macro-level patterns not reducible to, nor explicitly encoded in, the components' individual specifications [14]. Both terms apply to this system, but sloppily applied they explain nothing; the value lies in stating precisely *what* is designed and *what* emerges.

Designed are the local rules: the reward terms a drone experiences (all computable from its own transition), the fusion rule, the fault process and the training procedure itself. *Not designed*, that is nowhere represented in code or in weights as an explicit specification, are the team-level regularities the trained system is expected to exhibit: the early-episode radial dispersion from the shared spawn corner; the partition of the map into implicit, identity-correlated search sectors; the complementarity of coverage, meaning low overlap between drones' swept regions; and the re-absorption of a dead drone's sector by its survivors. Each of these is a macro-pattern over the joint trajectory of the team, produced by four independent evaluations of one function that has never been trained on, or even given access to, any team-level objective, each drone's inputs remaining strictly local throughout.

The classical checklist for self-organized coordination [7], namely multiple interacting units,

local information only, no external director, positive and negative feedback, is met, with the mechanisms identifiable one by one: negative feedback is the repulsion from consumed regions (revisit penalty, spent exploration bonus, collision penalty) that spreads the team out; positive feedback is the attraction created when the target's location enters the shared map and propagates through fusion, collapsing the team's search into a pursuit. The balance of the two is exactly what the policy has had to learn.

Two honest qualifications distinguish this from natural self-organization. First, the organization is *trained into* the system: the attractors of the collective dynamics were shaped by fifty thousand episodes of gradient descent against a designer-chosen reward, which is self-organization at execution time and teleology at training time. Second, these team-level regularities have the status of *design intent with partial quantitative support*: of the three signatures defined as their empirical test, Section 7.6 delivers coverage complementarity and leaves the two trajectory-level ones (dispersion over time, post-fault re-covering latency) to future instrumentation.

6.7 Fault tolerance, openness, and resilience

Multi-agent systems are routinely credited with robustness through redundancy and decentralization; this project turns that slogan into a concrete, testable mechanism. Three layers of the design implement it.

Structural redundancy. Every drone is a complete, interchangeable searcher: same policy, same sensing model, same authority (none over the others). The task requires *some* drone to reach the target, not any particular one, so no individual's loss is qualitatively worse than another's; combined with the environment-level guarantees that wrecks neither block corridors nor cost reachability, any single survivor retains in principle the ability to complete the mission alone, degraded in time but not in kind. This is the redundancy-based robustness familiar from biological collectives [7], engineered deliberately.

A weak form of openness. The team's composition changes at runtime: an episode that begins with four agents may end with one. In multi-agent terms the system is *semi-open*: departures occur unannounced, but no agent ever joins. Even this weak openness has real consequences. Nothing in any drone's machinery assumes a fixed team size (the policy is conditioned on team size and believed alive count, and per-agent structures are allocated per episode), and no drone's correct functioning depends on any other drone existing. The design would extend to arrivals, the context and fusion machinery being indifferent to *which* drones exist, though the trained policy has never experienced them: a gap between the architecture's openness and the policy's that is easily conflated.

Epistemic decentralization: acting on belief, not knowledge. The most distinctive layer is informational. No drone is told the team's true state: each maintains $\mathcal{C}_i^t$, the crashed teammates it knows of, and acts on the belief $\hat{n}_i^t = N - |\mathcal{C}_i^t|$. This is a genuine, if minimal, epistemic structure: a private, first-order factual belief state, updated by two canonically distinct sources, direct observation (entering beacon range or sight of a crash) and testimony (fusion with a better-informed teammate), with the propagation dynamics of a gossip process, fault knowledge spreading through the chance encounters of the communication graph at a rate governed by r_{comm} and by how the policy moves the drones. The soundness property established earlier, that $\hat{n}_i$ never undercounts the living, has a behavioural face: a drone's error is always in the direction of *overestimating* its team, continuing to rely, for a while, on a searcher that no longer exists. The interval between a fault and a given survivor's discovery of it is thus the system's window of miscoordination, and its cost is an empirical question that Section 7.4 takes up. The design decision worth defending here is the refusal of the tempting alternative, a global fault

oracle broadcasting team state, which would be simpler and would quietly reintroduce exactly the centralized, distance-insensitive information channel whose absence defines the problem.

Note finally how the three layers compose: redundancy makes survivors *sufficient*, epistemic decentralization makes them eventually *aware*, and context conditioning makes them *adaptive*, since $\hat{n}_i$ feeds the policy, so discovering a death is not merely bookkeeping but an input that licenses recalibrating the division of labour. Fault tolerance is not a module in this system; it is a property of the composition.

6.8 The autonomy spectrum

Autonomy is graduated, not binary, a lesson institutionalized in the level taxonomies of applied domains, from driving automation's six levels [44] to the human-delegation hierarchies of military doctrine [53]. Placing this system honestly on that spectrum matters twice: once for conceptual hygiene, and once because the placement determines which ethical questions Section 8 must answer.

The distinction that organizes the spectrum is between *automatic* systems, executing pre-specified stimulus-response rules however elaborate, and *autonomous* ones, which select and adapt their own courses of action toward a goal under conditions their designers did not enumerate. The drones sit unambiguously on the autonomous side: no trajectory, sweep pattern or contingency table exists anywhere in the system, the mapping from situations to actions was learned, and it adapts online to circumstances (team size, sensing regime, obstacle layout, teammate deaths) that vary per episode and per turn. The BFS auto-navigation override is the clarifying foil: *that* is an automatic component, a fixed and verifiable procedure, and Section 7 uses it precisely because its behaviour, unlike the policy's, is fully explainable.

Yet the drones' autonomy is strictly *executive and operational*, not motivational. They choose their actions; they do not choose, weigh, reformulate or abandon their goal, which was fixed by the designers, half at design time in the reward function and half at training time in its distillation into values, with no runtime process able to revise it. In delegation terms [53]: the human specifies *what* (find the target) and every constraint on *how* (the reward's economy of time, collision and coverage); the system autonomously resolves everything below that line and nothing above it. Two boundary facts make the point concrete. The drones cannot refuse the mission or trade it off against anything, there being nothing else in their value system; and the capability that looks most like initiative, re-planning the division of labour when the team shrinks, is itself goal-subordinate adaptation triggered by a belief update, in service of the same fixed objective.

This placement, with high operational autonomy, zero motivational autonomy, no runtime human in the loop and an opaque learned decision function, is exactly the profile for which accountability questions are sharpest: the system's behaviour is neither predictable enough to certify like an automatic device, nor deliberate enough to interrogate like a reasoning agent. Section 8 takes up precisely this configuration.

6.9 Learning in the multi-agent system

Parameter sharing [17] places this design at a specific point of the CTDE continuum [27, 31]: training-time centralization consists *only* of pooling experience into one replay buffer and one set of weights, with no centralized critic, no access to teammates' observations or actions, and no global state anywhere in the loss. Every transition that trains the network is something some individual drone actually experienced, and execution is correspondingly clean, since the batched action selection is a wall-clock optimization with no informational content. The deployment artefact is a fully decentralized policy in the strictest sense. The same choice disposes of two MARL classics at once: the *symmetry* of a shared policy at a shared spawn is broken by the identity entry of the context vector, and *scalability* follows from never materializing the joint anything, since sequential execution plus per-drone argmax means the joint action space whose exponential growth motivates much of the literature simply never appears, team size entering

as a conditioning scalar rather than a dimensionality. What sharing does *not* remove is the moving-target problem, each drone's optimal behaviour still depending on teammates' evolving behaviour, and here the design accepts the standard empirical wager of parameter-shared MARL: that co-evolution of one policy converges more gracefully than co-evolution of N.

The platform as a multi-agent laboratory. A final identity shift is worth recording: the same artefact that trains the policy is, unchanged, an agent-based simulation platform [5], a micro-level model whose macro-behaviour is studied by controlled manipulation. The seeded, parameterized evaluation harness is methodologically identical to a simulation experiment design: hold the population of instances fixed, vary one factor (p_{fault}, r_{comm}, team size), and measure macro-level responses. In this reading Section 7 is an agent-based "what-if" study of the trained collective: what happens to a searching society as its members become mortal.

6.10 What a richer multi-agent design would add

The system's multi-agent machinery is deliberately minimal, and the analysis above should not read as a claim that minimal is optimal. It is worth stating what each classical layer this design omits would offer, and what it would cost, both as intellectual honesty and as a roadmap.

Explicit communication protocols [11] would let drones exchange intentions rather than facts ("I will sweep the north-east quadrant"), enabling explicit task allocation and with it verifiable non-overlap guarantees instead of statistically-learned dispersion. The cost is the machinery this design conspicuously avoids: message semantics, conversation state, commitment handling, and the robustness of all of it to the death of either partner mid-protocol, a non-trivial concern in precisely this problem.

Deliberative architectures [43] would make goals and plans explicit and inspectable. The gain is not primarily performance but *explainability*: a belief–desire–intention drone answers "why did you go north?" with an intention trace, where the present drone can answer only with a Q-value, a difference whose ethical weight Section 8.5 makes precise. The cost is the classical one: hand-authored plan libraries in a domain whose main difficulty is exactly what resists concise symbolic specification.

Organizational structure, meaning roles, formations and an explicit division of the map, would replace learned conventions with designed ones, gaining predictability and losing the seamless re-adaptation that makes the fault response work: a role structure must be re-formed when its occupants die, whereas the learned convention degrades continuously with $\hat{n}_i$.

Programmable coordination media [13, 36] offer perhaps the most natural upgrade path, since the system already has a coordination medium (the fused map) with a fixed policy (max-merge). Making that medium programmable, with a fusion rule that could age knowledge, prioritize target and crash information under bandwidth limits, or maintain provenance, would relocate coordination intelligence from the opaque policy into an inspectable artefact, the move the artefact literature explicitly recommends [37].

The common thread is a single trade-off, and it is the honest summary of this section: every engineered layer buys transparency, verifiability and designer control, and pays in brittleness and specification burden; the learned-minimal design buys adaptivity across regimes that would each demand their own specification, and pays in opacity. This project sits knowingly at the learned end. The experimental section measures what that position buys; the ethical section confronts what it costs.

7 Experimental Evaluation

This section evaluates the single shared policy trained in Section 5. The questions are deliberately those the multi-agent reading of Section 6 raised and left open: does one policy really generalize across the randomization axes it was trained over; does it degrade gracefully as random faults

remove members of the team, the project's defining scenario; and are the coordination regularities claimed as emergent visible in aggregate behaviour? We first fix the protocol and the comparison points, then take the axes one at a time, and close with the limitations that bound every claim made here.

7.1 Experimental setup

Policy under test. Unless stated otherwise, all results use the final checkpoint of the 50,000-episode run, evaluated *greedily* ($\varepsilon = 0$), so faults, when present, are the only source of stochasticity in an episode. The learning curve below additionally revisits the periodic checkpoints of the same run.

One-factor-at-a-time protocol. The policy was randomized over five environment factors simultaneously, so the natural evaluation is a controlled *one-factor-at-a-time* (OFAT) sweep: we vary one factor at a time and hold the others at a fixed *reference operating point*, namely team size $N = 3$, vision $r_{\text{vis}} = 3$, communication $r_{\text{comm}} = 5$, obstacle density $\rho = 0.2$, step budget $T = 200$ and no faults, which is the configuration file's default operating point and sits near the centre of the training ranges of Table 1. This maps each axis without the exponential cost of a full grid; the two interactions we do examine jointly are reported as heatmaps in Section 7.6. Each configuration point is evaluated on the *same* bank of 500 seeds (learning-curve and heatmap points use 300); a seed fixes the obstacle layout, target, spawn corner and, under faults, the entire fault sequence, so points differ only in the factor being swept. In every plot the range actually seen in training is drawn on a white background and the extrapolation beyond it (*out-of-distribution*, OOD) on a shaded one. The step budget is the one axis to which this distinction does not apply: T was never randomized, training having fixed $T = 200$ throughout, so every other value on that axis is off-distribution and no OOD *probe* can be singled out.

Metrics. We report the task-level metrics defined in Section 3.6, plus a standard path-quality measure. *Success rate* is the fraction of episodes reaching the target within $T \cdot N$ agent-turns, with a Wilson 95% confidence interval for the binomial proportion. *Time-to-find* is the number of *rounds* elapsed at success, averaged over successful episodes only. The simulator advances in agent-turns and every drone consumes one turn per round, wrecks included since a broken drone's no-op still costs its turn, so the raw count is divided by N. This reports elapsed mission time rather than accumulated drone effort, and puts the metric in the same unit as the step budget T, against which it would otherwise be incomparable. *Coverage* is the fraction of free cells the team has discovered at episode end; *overlap* is the fraction of team-visited cells visited by more than one drone, and *revisits* the mean per-drone re-entries, the two redundancy signatures promised in Section 6.6. *SPL* (Success weighted by Path Length) [1] weights each success by the ratio of the optimal spawn-to-target distance to the finder's actual path length, rewarding *short* solutions and scoring failures zero; it is our single scalar for path quality.

A note on episode return. The shaped return of Table 2 is reported alongside these metrics but is *not* used to rank configurations, and under faults it is actively misleading: a drone that breaks stops paying the per-turn step penalty, so removing team members can *raise* the mean return even as fewer targets are found. This is precisely the shaping-versus-task discrepancy that motivated the success-dominant checkpoint rule of Section 5.3; we therefore read success rate, not return, as the primary metric throughout.

At the reference operating point the policy reaches the target in 87.6% of episodes (Wilson CI $[84.4, 90.2]$), discovering 53.7% of the free space with an SPL of 0.541: the anchor every one-factor sweep below passes through. The learning curve of Figure 9 ends slightly lower, at 85.0%, for the same checkpoint at the same operating point; the two reconcile exactly, that curve being computed on the 300-seed prefix (2000–2299) of this 500-seed bank. One plotting convention

Axis	In-distribution sweep	Success [%] (in-dist.)	OOD probe	Success [%] (OOD)
Team size N	$1 \to 4$	$71.0 \to 89.2$	5, 6	89.6, 90.4
Vision r_{vis}	$1 \to 6$	$70.6 \to 87.2$	7, 8	87.0, 85.6
Comm. r_{comm}	$2 \to 12$	$87.4 \to 89.0$	0, 14, 16	87.6, 88.0, 89.6
Density ρ	$0.30 \to 0.10$	$53.4 \to 96.2$	0, 0.35, 0.40	99.0, 34.6, 24.2
Budget T †	$50 \to 300$	$49.0 \to 89.8$	—	—

Table 4: Generalization summary (greedy policy, 500 seeds per point). Each row sweeps one factor with the others held at the reference operating point. "Success (in-dist.)" reports the endpoints of the in-distribution sweep in the stated direction; the OOD probes extrapolate beyond the training range on either side. Team size, vision and communication all extrapolate without a cliff, including beyond their trained maxima; obstacle density is the one axis that governs difficulty, and the only one whose OOD extrapolation collapses, though a large part of that collapse is the instance distribution rather than the policy (Section 7.3). The obstacle-free case $\rho = 0$ lies below the trained floor of 0.10 and is a benign extrapolation at 99.0%. † The step budget was never randomized (training fixed $T = 200$), so this row is an operating-condition sweep rather than an in/out-of-distribution one and admits no OOD probe.

holds throughout: bounded panels are drawn on axes fitted to the data plus the confidence band, so that trends of a few points remain legible; map coverage is the exception and keeps the full 0–100% range, its variation along the flatter axes being comparable to the sampling noise at 500 seeds.

7.2 Baselines and reference points

Two comparison points frame the results, neither of which is an upper bound.

The single drone (value of cooperation). The $N = 1$ end of the team-size sweep is the natural "no cooperation" baseline: the same policy, deprived of any teammate to coordinate with. It isolates what the team buys over a lone searcher and is discussed with the sweep below.

BFS auto-navigation (a hybrid skyline). The evaluation-only override of Section 4.7 replaces the learned action by the first step of a breadth-first shortest path, on the drone's *own known map*, whenever that drone already knows where the target is. It is a hybrid, since the learned policy still does all the *searching* and the planner only optimizes the final *approach* once the target has been found, and it is not an oracle: it plans on partial knowledge, and it cannot help an episode in which no drone ever sees the target. We use it to separate two competences the success rate conflates: finding the target, and travelling to it efficiently. A gap in favour of auto-nav localizes a weakness in the policy's homing, not in its search.

A uniform-random controller was not run as a separate condition: on a 32×32 map with a single target and a step budget its success is a floor far below every operating point reported here, and the single-drone and auto-nav references are the informative comparisons.

7.3 Generalization across randomization axes

Table 4 summarizes all five OFAT axes; we read them in three groups.

Team size: cooperation helps, and extrapolates. Adding drones raises success monotonically, from 71.0% for a lone searcher to 89.2% at $N = 4$ (Figure 6), and, the result worth stressing, the curve does not break when the team grows *past* any size seen in training: $N = 5$ and 6 reach 89.6% and 90.4%. A single shared network, conditioned only on the scalar team size and a per-drone identity, thus drives teams larger than it ever trained on without retraining or architectural change, which is the concrete pay-off of parameter sharing plus context conditioning argued in Section 6.9. SPL climbs alongside success ($0.33 \to 0.58$ over $1 \to 4$, continuing to 0.64

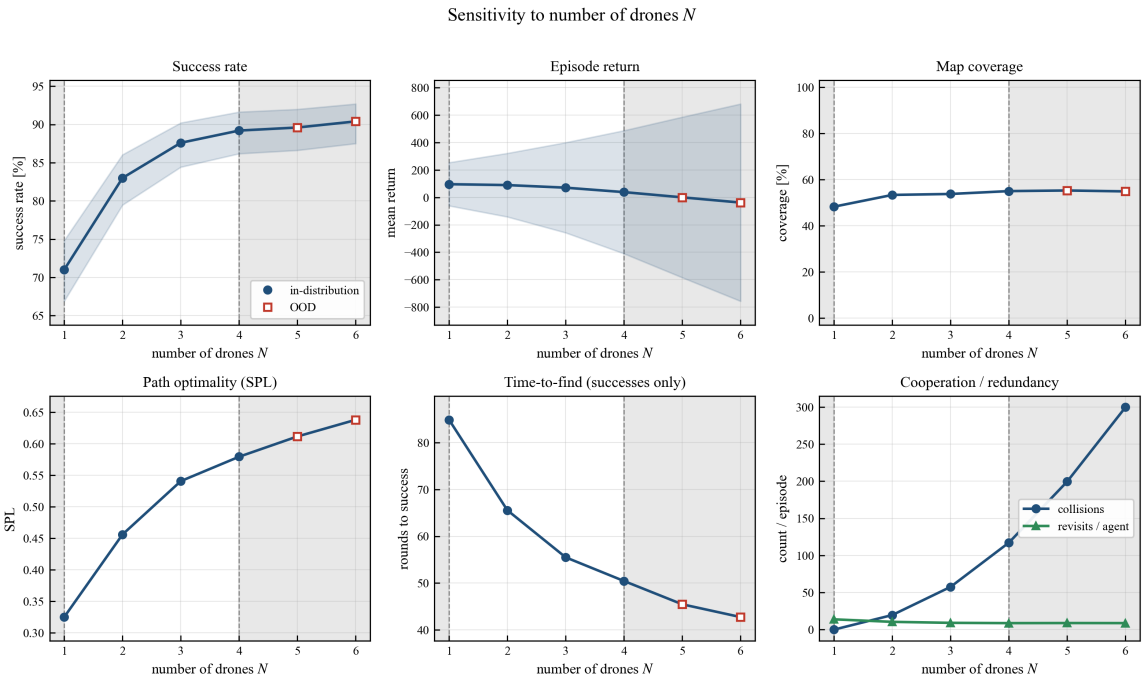

Figure 6: Sensitivity to team size N. Blue circles are in-distribution ($1 \le N \le 4$), red squares the OOD extrapolation $N = 5, 6$, shaded bands the Wilson 95% CI (success) or ± 1 standard deviation (return).

at $N = 6$): more searchers means the target is typically found by whichever drone is already nearest, shortening the winning path.

Larger teams are also genuinely *faster*, and not merely more likely to succeed. Mission time falls from 84.9 rounds for a lone searcher to 55.5 at $N = 3$ and 50.4 at $N = 4$, continuing to 42.7 at $N = 6$. The speed-up is real but markedly sublinear: 1.53× at $N = 3$ and 1.99× at $N = 6$, a parallel efficiency falling from 51% to 33%. Doubling the team from three drones to six therefore cuts mission time by a further 23% rather than halving it again, which is the quantitative form of the coordination overhead the next paragraph makes concrete.

Two costs temper the picture, both visible in Figure 6. Mean return *falls* as the team grows, since more drones each pay the step penalty, which is why return is not our ranking metric. And same-cell collisions rise steeply with N, an unavoidable consequence of a shared spawn corner and a bounded map, where more bodies contend for the same early cells. Congestion is precisely the mechanism behind the diminishing returns reported above: an added searcher spends an increasing share of the episode contending for, and re-sweeping, ground the team already holds. The two redundancy statistics separate the individual from the collective view of this: per-drone revisits fall and then flatten, each drone individually more efficient, while team overlap rises (18.5% → 37.0% from $N = 2$ to 6), the team collectively more wasteful. Coverage confirms the ceiling: it plateaus at 53.7–55.2% from $N = 3$ on, so beyond that point extra drones mostly duplicate each other's ground rather than extend the swept region.

Vision and communication: robust, with a telling asymmetry. Neither sensing axis has a cliff, in or out of distribution (Table 4), but the two are robust in quite different ways. Vision genuinely helps, and on every quality metric at once (Figure 7): from $r_{\text{vis}} = 1$ to $r_{\text{vis}} = 6$ coverage rises from 46.5% to 62.9% (reaching 69.5% at the OOD radius $r_{\text{vis}} = 8$), SPL nearly doubles (0.33 → 0.63), and mission time falls from 81.5 rounds to 43.0, close to a halving. What saturates is the *find rate*: success climbs steeply out of myopia (70.6% at $r_{\text{vis}} = 1$ to 87.6% at $r_{\text{vis}} = 3$), peaks at 88.2% around $r_{\text{vis}} = 4$, then drifts gently down in and out of distribution alike, because past that point the bottleneck is deciding *where* to go rather than seeing farther. A wider window therefore keeps buying speed and path quality long after it has stopped buying missions.

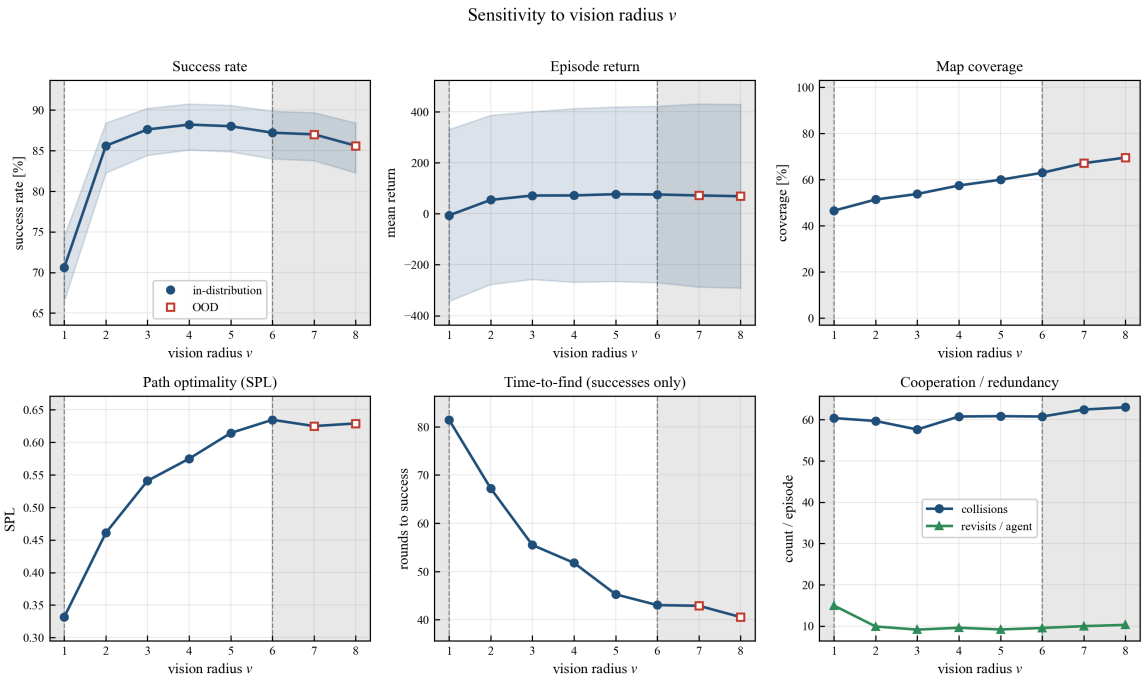

Figure 7: Sensitivity to vision radius r_{vis}. Blue circles are in-distribution ($1 \le r_{\text{vis}} \le 6$), red squares the OOD probes $r_{\text{vis}} = 7, 8$, shaded bands the Wilson 95% CI (success) or ± 1 standard deviation (return).

Communication is the more interesting axis, because there the success curve really is flat: 87–90% across the entire range, and severing fusion outright ($r_{\text{comm}} = 0$, an OOD probe) still yields 87.6%. What communication buys is efficiency, and it buys a lot of it: between $r_{\text{comm}} = 0$ and $r_{\text{comm}} = 10$, SPL rises from 0.492 to 0.578 and mission time falls from 61.7 rounds to 49.6, gains of 17% and 20%, while redundancy drops as shared maps let drones avoid re-covering each other's ground. The asymmetry, then, is that vision governs both *whether* and *how well* the team searches, while communication governs only *how well*. We take the latter up directly as an ablation in Section 7.5, as it bears on the coordination analysis of Section 6.4.

Density and budget: the difficulty axes. Obstacle density is the one factor that dominates the task (Figure 8). Success falls smoothly from near-perfect in open fields (99.0% at $\rho = 0$, 96.2% at the trained floor $\rho = 0.10$) to 53.4% at the trained ceiling $\rho = 0.30$, and then collapses out-of-distribution to 34.6% and 24.2% at $\rho = 0.35, 0.40$, the only axis whose extrapolation fails outright. The mechanism is legible in the companion panels: as clutter rises, corridors narrow, drones are funnelled together (collisions and per-drone revisits climb sharply), coverage drops, and SPL falls as detours lengthen the winning path. Time-to-find is the one panel there not to read as a difficulty signal: averaged over successful episodes only, its *drop* at the OOD densities (66.4 rounds at $\rho = 0.30$ against 21.1 at $\rho = 0.40$) is survivorship, since at 24.2% success the few episodes that still finish are those whose target happened to lie near the spawn corner.

Part of this collapse, however, is not a policy failure at all. As noted in Section 3.1, reachability is guaranteed only from the reference corner $(0, 0)$ while the team spawns on a uniformly chosen one, so a fraction of instances have no solution to find. Running a breadth-first search from the actual spawn on the very same 500 seeds gives the *solvable* fraction, and hence the highest success rate any controller could reach:

ρ	0	0.10	0.15	0.20	0.25	0.30	0.35	0.40
Solvable [%]	100.0	98.6	95.8	91.4	80.6	65.4	45.6	30.4
Success [%]	99.0	96.2	94.4	87.6	75.6	53.4	34.6	24.2
Of achievable [%]	99.0	97.6	98.5	95.8	93.8	81.7	75.9	79.6

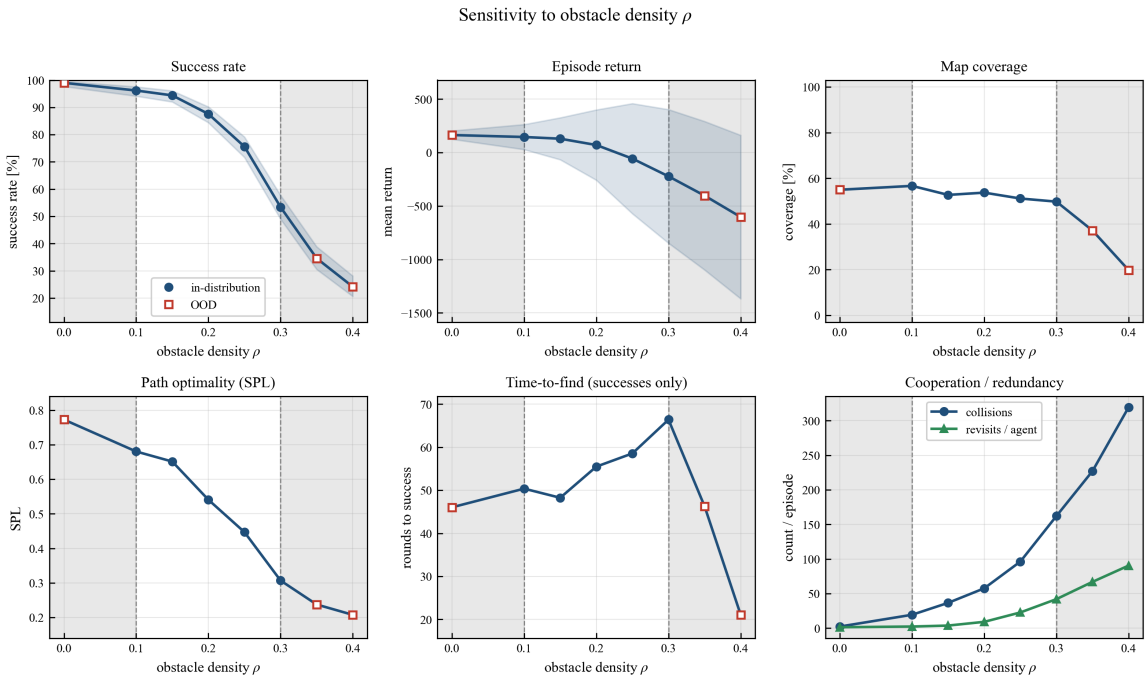

Figure 8: Sensitivity to obstacle density ρ, the one axis that governs difficulty. Shaded points ($\rho = 0$ and $\rho > 0.30$) are the OOD extrapolation, the trained range being $[0.10, 0.30]$. Note that time-to-find, averaged over successful episodes only, is a survivorship statistic at those densities rather than a difficulty signal.

Read this way the axis tells a more honest story: most of the drop is the instance distribution becoming unsolvable rather than the policy degrading, and even at the far OOD point the team still finds four targets in five of those that *can* be found. Genuine degradation is present, and is concentrated between $\rho = 0.25$ and $\rho = 0.30$, where the achievable share falls from 94% to 82%. The raw success rate remains the headline number, being what a deployed system would experience, but the OOD collapse of this axis is substantially structural and milder than the raw curve suggests.

The step budget behaves as expected of a pure time constraint: success rises with T (49.0% at $T = 50$ to 89.8% at $T = 300$, Figure 14) with diminishing returns, since a generous budget only helps episodes that were close to succeeding anyway.

Learning curve. Re-evaluating the periodic checkpoints at the reference operating point (Figure 9) shows success rising to a plateau around 79–86% by roughly episode 25,000 and holding, with SPL continuing to inch up afterward: the policy keeps refining *how* it reaches targets after the *whether* has largely converged. The intermediate dip around episodes 17,500–22,500 (success 68–70%) falls in the stretch where exploration has long since floored, ε hitting 0.05 at episode 5,990, while the learning rate is still 67–82% of its peak on the cosine schedule: large updates against a nearly greedy, self-generated state distribution. It recovers without intervention as the annealing proceeds. The near-greedy evaluation logged online during the run itself ($\varepsilon = 0.05$ on a different, 20-episode bank) also dips in this region, at episodes 16,000 and 21,000, but that bank is far too small to corroborate the timing: at 20 episodes a 0.55 against 0.70 difference is well inside the sampling noise, and comparable dips appear elsewhere on the same curve. This curve is the greedy, 300-seed complement to the online training scalars of Figure 5.

7.4 Fault robustness

This is the project's headline experiment: how does a policy trained over $p_{\text{fault}} \in [0, 0.003]$ behave as drones are actually lost mid-mission? Figure 10 and Table 5 sweep the per-turn fault probability from the fault-free reference out to $p_{\text{fault}} = 0.01$, more than triple the trained maximum: a regime

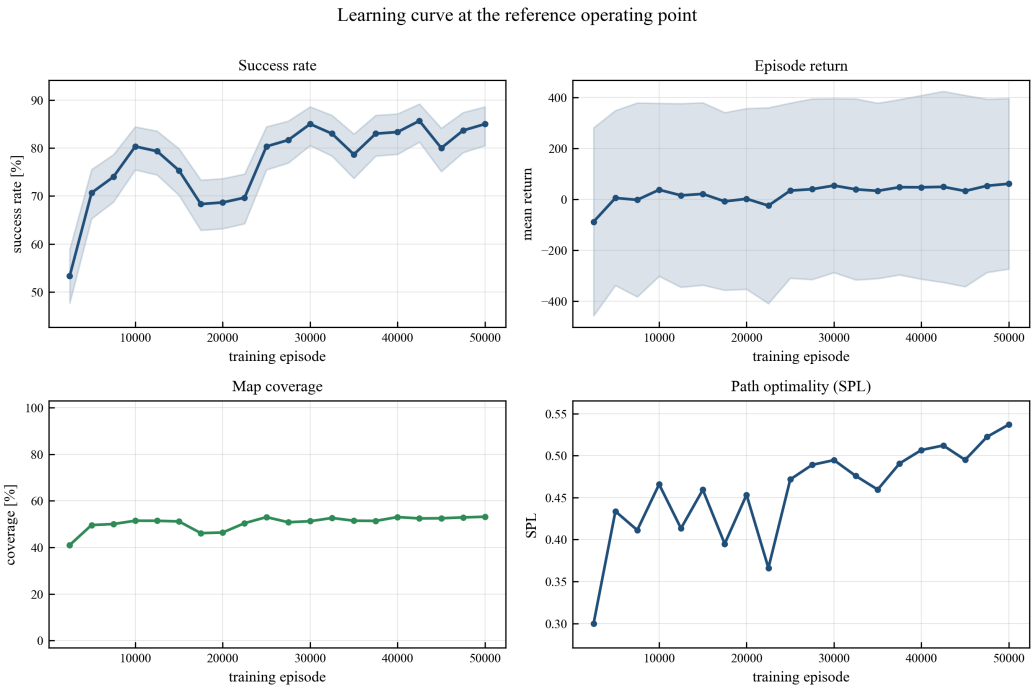

Figure 9: Learning curve: every periodic checkpoint of the final run, evaluated greedily at the reference operating point (300 seeds per checkpoint).

p_{fault}	**Success [%] policy**	**Success [%] + auto-nav**	**SPL (policy)**	**Drones lost / episode**	**Regime**
0.000	87.6	89.8	0.541	0.00	in-dist.
0.001	86.0	88.4	0.532	0.22	in-dist.
0.002	84.4	87.2	0.517	0.42	in-dist.
0.003	82.0	84.8	0.505	0.58	in-dist.
0.005	77.2	80.2	0.480	0.91	OOD
0.010	66.2	68.4	0.404	1.51	OOD

Table 5: Fault robustness (500 seeds per point, reference operating point otherwise). Across the entire trained range the team loses at most ~ 6 percentage points of success while shedding on average more than half a drone; degradation stays graceful well past the trained maximum. The BFS auto-nav hybrid is a consistent ~ 2–3 points better, locating a small residual inefficiency in the policy's final approach rather than in its fault handling.

in which, over 200 rounds, a drone's survival probability is only $0.99^{200} \approx 0.13$ and, at the reference team size $N = 3$, roughly 1.5 drones are lost per episode on average.

Graceful degradation. The central result is that degradation is gentle and monotone. Across the *entire* trained range the success rate falls only from 87.6% to 82.0%, under six points, even though by $p_{\text{fault}} = 0.003$ the team is losing, on average, 0.58 of its three drones per episode. Pushed to the 3.3× OOD point $p_{\text{fault}} = 0.01$, success degrades to 66.2% with 1.51 drones lost: the policy sheds, on average, *half* the team and still completes two missions in three. There is no cliff. This is the redundancy argued in Section 6.7 made quantitative, since any surviving drone can complete the search, so the loss of some drones removes capacity rather than the mission; and it holds under a fault rate the policy was never trained on, evidence that the decentralized fault handling generalizes rather than memorizing the training range.

Return rises while success falls. Table 5 and the training scalars make the shaping artefact concrete: mean return *rises* across the whole trained fault range (71.2 → 86.4, peaking at 89.8 before ever-shorter episodes pull it back down) because wrecked drones stop incurring step

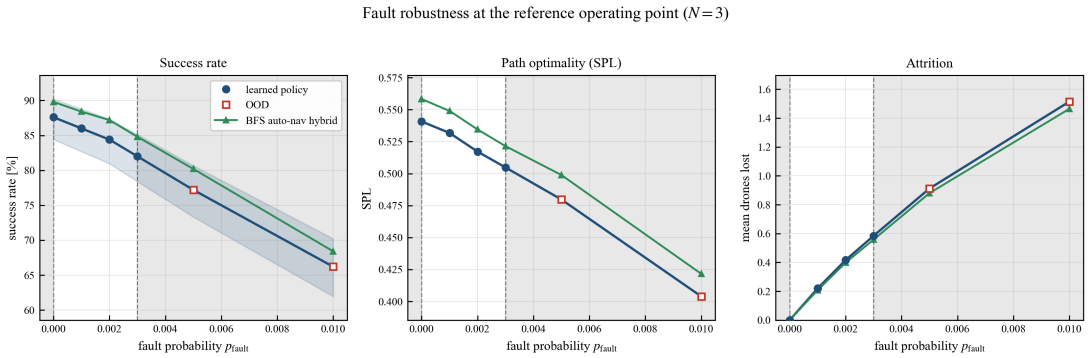

Figure 10: Fault robustness. Learned policy (blue) versus the BFS auto-nav hybrid (green); the shaded region is OOD ($p_{\text{fault}} > 0.003$). Panels: success, SPL and the mean number of drones lost per episode.

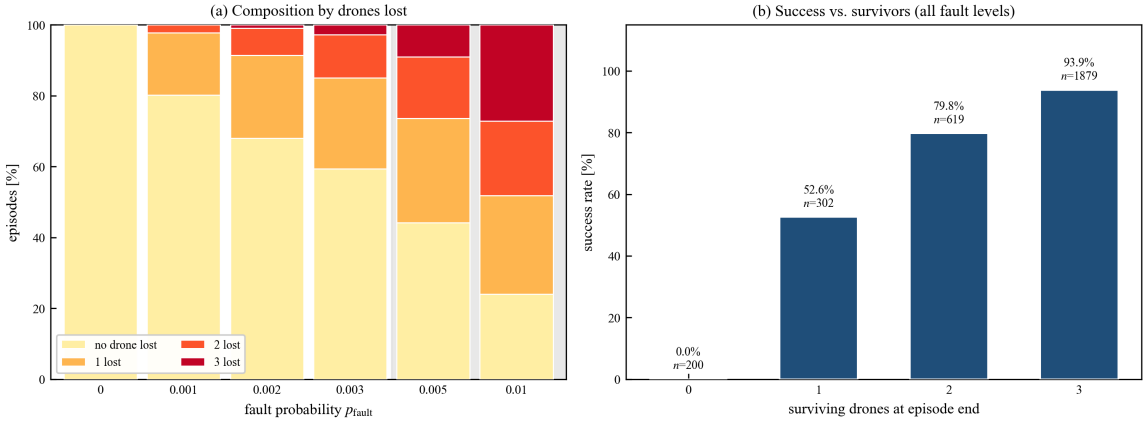

Figure 11: Opening up the fault aggregate (500 seeds per fault level, reference operating point, $N = 3$). *(a)* Composition of episodes by number of drones lost, per fault level. *(b)* Success rate conditioned on the number of survivors, pooled over all fault levels; the zero-survivor bar is 0% by construction, an all-dead team truncating the episode.

penalties. Read naively, the reward says faults *help*; read correctly, success says they hurt, slowly. The experiment is thus also a validation of the success-dominant model-selection rule of Section 5.3: had we selected checkpoints on return, we would have preferred policies that let drones die quietly.

What the aggregate hides: how many drones actually die. The mean loss counts of Table 5 average over very different episodes, so Figure 11 opens them up. Panel (a) shows the full composition: even at the trained maximum $p_{\text{fault}} = 0.003$, a majority of episodes (59%) still lose *no* drone, and total wipe-outs are rare (3%); at the 3.3× OOD point the picture inverts, with only 24% of episodes fault-free and 27% losing the entire team. Panel (b) conditions success on how many drones survived, and this is where the graceful-degradation claim becomes mechanical rather than rhetorical. A team that finishes intact succeeds 93.9% of the time; losing one drone costs 14 points (79.8%), and even a lone survivor still finds the target in 52.6% of episodes. That last figure sits 18 points below the 71.0% of a team that was *born* alone, which is the honest reading of the cost of attrition: a sole survivor is not equivalent to a solo searcher, having lost part of the budget to a team that no longer exists and inheriting a partially swept map from a worse position. It remains, however, a degradation of degree rather than of kind. The 0% bar at zero survivors is definitional, not a policy failure: an all-dead team truncates the episode, so those 200 episodes are lost to the environment rather than to the policy.

Where the small gap lives. The auto-nav hybrid tracks the policy closely and a constant 2–3 points above it, at *every* fault level (Figure 10). Because auto-nav changes only the homing

phase, this near-constant offset says the policy's residual weakness is a slightly sub-optimal final approach, not a failure to handle faults, which would have widened the gap as p_{fault} grew. Fault tolerance is a property the learned policy already has; efficient homing is the part a planner can still improve.

The belief-lag question. Section 6.7 identified the interval between a fault and a survivor's discovery of it, governed by r_{comm}, as the system's "window of miscoordination". Measuring it directly needs a joint $p_{\text{fault}} \times r_{\text{comm}}$ sweep with per-step trajectory logs, which the present episode-summary harness does not record; an indirect bound is available meanwhile. Since success is nearly insensitive to r_{comm} even without faults, what fusion spreads being on this task largely redundant with what a drone sees for itself, the additional value of faster fault-news propagation is likely second-order for the find rate, though it may matter more for path quality.

7.5 Ablation: the role of communication

The communication axis doubles as a clean ablation, because $r_{\text{comm}} = 0$ removes map fusion entirely without retraining or changing anything else. Figure 12 isolates it, and the result splits cleanly in two. Fusion is not what lets the team *find* the target: success is 87.6% with communication severed against 87–90% over the whole range, a spread well inside the confidence intervals. It is what lets the team find it *well*. Switching fusion on is worth between a seventh and a fifth of the team's efficiency on every measure the harness records. Between $r_{\text{comm}} = 0$ and the trained maximum $r_{\text{comm}} = 12$, SPL rises from 0.492 to 0.587 (+19%), per-drone revisits fall from 9.9 to 8.6 (−13%) and inter-drone overlap from 26.6% to 21.0% (−21%), while mission time drops from 61.7 rounds to 49.6 at its best (−20%). For a mechanism that costs nothing beyond being within range of a teammate, and that no reward term ever instructed the policy to exploit, taking something close to a fifth off the cost of a mission is a substantial return.

The reading matters for the multi-agent analysis. Communication is not what *creates* coordination here: with the channel cut the team still disperses and still succeeds, which says that the dispersion itself is carried by the *environment-mediated, stigmergic* channel of Section 6.4, drones spreading out because regions they or their teammates have swept are made unrewarding. What fusion adds is efficiency, and it adds it through the same artefact, the accumulated map, rather than through any protocol: two coordination channels sharing one representation, complementary rather than redundant, which is as concrete an instance of the coordination-without-protocols theme of Section 6 as the system offers. The honest caveat runs the other way from the usual one: because the find rate is insensitive here, this sweep *understates* what fusion would be worth on a task that genuinely required information sharing (a target visible only from afar, say, or heterogeneous sensing), where it would have to carry success and not merely efficiency.

Two further ablations named in the analysis, removing $\hat{n}_i$ from the context vector and switching the shared terminal reward for an individual one, change the *training* objective and so require retraining a policy each; they are beyond the single-checkpoint scope of this evaluation and are noted as future work in Section 9.

7.6 Emergent coordination signatures

Section 6.6 claimed three measurable signatures of emergent division of labour and committed to testing them here: complementary coverage (low inter-drone overlap), inter-drone distance distributions over time, and post-fault re-covering latency. We report the first, which the episode-summary metrics capture directly, and are explicit about the two that require trajectory-level logging.

Complementary coverage is real and quantified. Across the team-size sweep, the overlap between drones' swept regions stays low: at $N = 4$ only 29.6% of team-visited cells are entered by

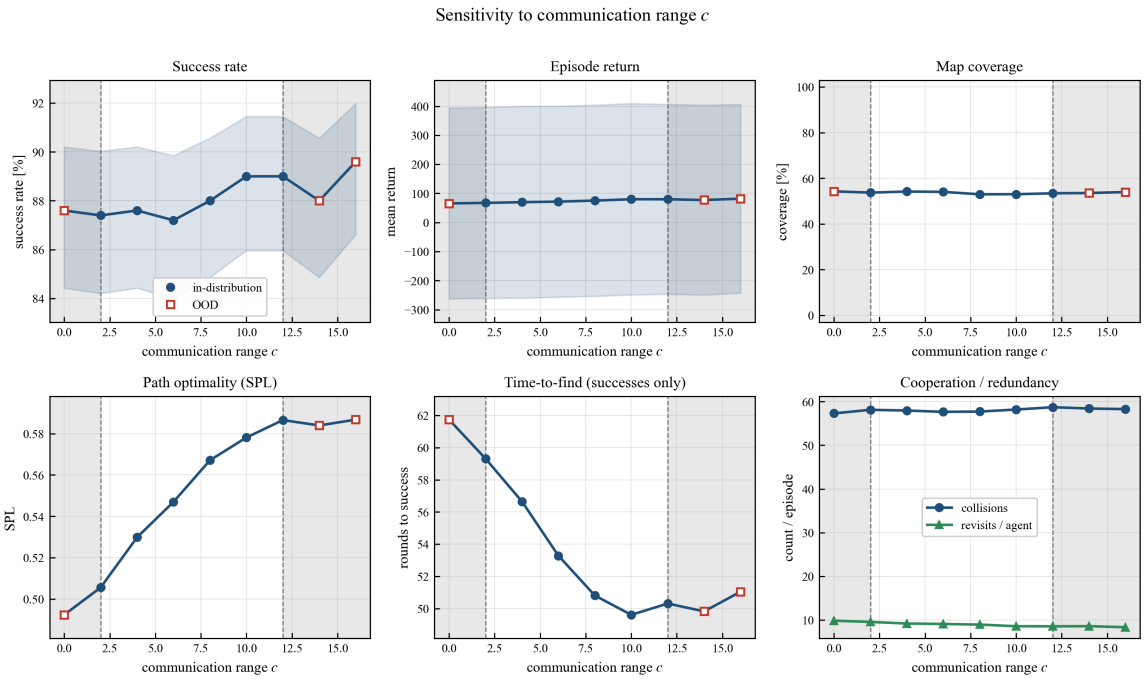

Figure 12: Communication ablation via the r_{comm} sweep. The leftmost point $r_{\text{comm}} = 0$ (OOD, shaded) fully disables map fusion; $r_{\text{comm}} = 14, 16$ are the OOD probes above the trained maximum.

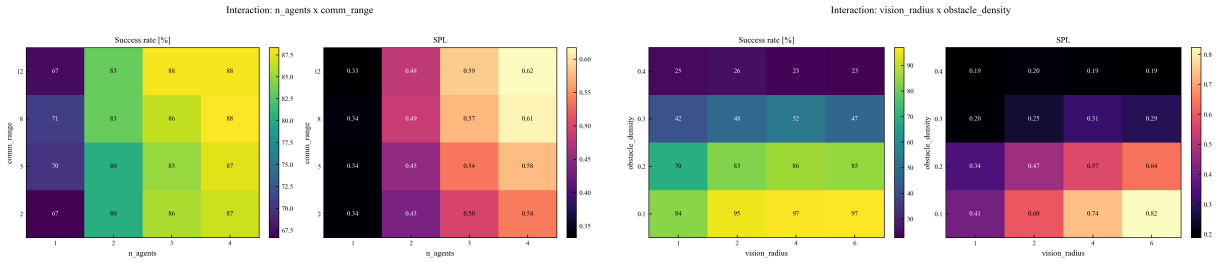

Figure 13: Two-factor interaction grids (300 seeds per cell): team size × communication range (left) and vision × obstacle density (right).

more than one drone, so roughly seven in ten cells are each the responsibility of a single searcher. More tellingly, *per-drone* revisits fall as the team grows, from 13.9 at $N = 1$ to about 8.8 from $N = 4$ on, where they flatten, even though total collisions rise: each drone individually wastes less effort re-treading its own ground as the map is partitioned among more searchers. This is exactly the complementarity predicted from local rules alone, no region ever being assigned, and it is the aggregate footprint of the identity-conditioned sector partition the shared policy learned.

Two interactions, read as heatmaps. Figure 13 confirms on 2-D grids what the OFAT sweeps imply marginally. Team size × communication (left) shows success governed almost entirely by N, with the r_{comm} direction nearly flat at every team size: the communication finding above, now visibly independent of team size. Vision × density (right) shows the reverse: density sets the achievable success band, and vision helps at low-to-moderate clutter, where seeing farther translates into faster coverage (the $r_{\text{vis}} = 1 \to 6$ gain peaks at +15.7 points at $\rho = 0.2$), while in dense clutter it is worth almost nothing, the extra sight lines being wasted on walls. The two dominant factors of the whole study, how *many* drones and how *cluttered* the world, are exactly the two that these interaction grids single out.

What we do not yet measure. The other two signatures, how pairwise inter-drone distances evolve within an episode (the radial-dispersion claim) and the latency with which survivors

re-cover a wrecked drone's sector, are properties of the *joint trajectory over time*, and the current harness records only end-of-episode aggregates; measuring them needs per-step position logging and a re-run of the fault sweep with trajectories retained. The emergence claims therefore rest, at present, on the coverage-complementarity signature above and on qualitative checkpoint inspection, and no further.

7.7 Discussion and limitations

The evaluation supports a bounded but clear claim: a single shared policy, trained with domain randomization and context conditioning, generalizes across team size, sensing and communication (including beyond its trained ranges), degrades gracefully under a fault process it only partially saw in training, and exhibits at least one measurable signature of emergent division of labour. The limits of that claim should be stated as plainly as the claim itself, since Section 8 leans on them.

Simulation and abstraction. Everything here is a 32×32 occupancy grid with four-connected moves, noiseless local sensing and instantaneous range-limited fusion. Continuous dynamics, sensor noise, localization error, communication latency and packet loss, each of which would stress the very mechanisms (map fusion, beacon detection) the policy relies on, are absent, and no sim-to-real transfer is claimed or tested.

The fault model is benign. Faults are i.i.d. across drones and turns, independent of state and action, and produce inert, non-blocking wrecks. Correlated failures, faults that block passage, and an *adversary* choosing when and whom to disable are all outside the model, and an adversary is exactly the case the security discussion of the next section must contend with.

Statistical, not formal, evidence. Success rates on 500 seeds with Wilson intervals bound sampling error, not worst-case behaviour: nothing here certifies how the opaque learned policy acts on a specific unseen instance, a gap Section 8.5 returns to. The OOD probes show graceful extrapolation on the axes tested but do not guarantee it in general, density already showing where extrapolation fails.

The instances are not all solvable. Reachability is guaranteed from the reference corner while the team spawns on a random one, so a density-dependent fraction of episodes has no solution to find. Every success rate here is therefore measured against a ceiling below 100%: the table above separates the two effects on the density axis, and the remaining sweeps, all run at $\rho = 0.2$, carry an unstated 8.6% floor of impossible instances. Seeding reachability from the drawn corner would remove the mismatch, at the cost of making every number reported here incomparable with the new ones.

Instrumentation gaps. The belief-lag / fault×comm interaction and two of the three emergence signatures were defined but not measured, for want of per-step trajectory logging, and the two training-objective ablations were scoped out for want of additional training runs. None of this undermines the results reported; they mark where the next iteration of the evaluation would sharpen them.

8 Ethical and Regulatory Analysis

A grid-world simulation processes no personal data, harms no one, and needs no licence. The analysis of this section is therefore explicitly *prospective*: it asks what would follow, ethically and legally, if the capability this project prototypes were embodied in real aircraft over real terrain. The exercise is not hypothetical hand-wringing. The task specification of Section 3, that is finding a hidden target in minimum time under partial observability with a team resilient to attrition, is the abstract core of application profiles ranging from the unambiguously beneficial to the gravest imaginable, and the technical analysis of Section 6, in particular the autonomy placement of Section 6.8, supplies exactly the vocabulary in which the ethical questions are posed. We proceed from the dual-use diagnosis, through the two European frameworks that would govern civilian

deployments, to the military scenario that escapes both, the responsibility problem underlying all of it, and a set of concrete design commitments.

8.1 A dual-use capability

Nothing in this system knows what its target *is*. The environment marks one cell; the policy learns to reach it quickly. Instantiate the target as a missing hiker and the system is a search-and-rescue asset [42]; as a wildfire front, an environmental monitor [25]; as a person of interest, a surveillance tool; as a military objective whose localization triggers engagement, a component of a weapon system. The capability is deployment-agnostic by construction, which is the precise sense in which it is *dual-use*: the same artefact serves civilian and military ends without modification, the classification hinging entirely on context of use.

What sharpens the point beyond the generic observation that search generalizes is that this project's two central technical results are, read adversarially, threat properties. Fault tolerance through redundancy and decentralized fault knowledge is exactly what makes a hostile swarm hard to defeat, resilience to random attrition and resilience to defensive fire being the same competence under different descriptions; and single-policy generalization across team sizes, sensing and communication regimes is robustness of the threat across countermeasure conditions. One cannot celebrate these properties in Section 6 and disown them here; the honest response is not denial but analysis: which uses are governed by what rules, where the rules run out, and what the designer can do unilaterally.

8.2 Data protection: the GDPR lens

Consider first the benign deployment: drones searching for missing persons over inhabited areas. Real perception is not an oracle bit but cameras and sensors over public and private space, and aerial imagery of identifiable individuals is personal data, so processing falls under the General Data Protection Regulation [8], or, for law-enforcement authorities acting as such, under Directive (EU) 2016/680. Consent is structurally unavailable, so processing must rest on another Article 6 basis (vital interests for rescue, public task for civil protection) under the discipline of purpose limitation; and *mission creep* is not an abstract worry for a system whose fused maps accumulate, by design, everything any team member has ever seen. At that scale, a multi-drone sweep of an inhabited area is also a textbook trigger for a mandatory data protection impact assessment (Article 35(3)(c)).

The architecture does, however, supply its own minimization argument, and it is the strongest ethical point this design can make. Every mechanism of Sections 3–4, whether coordination, fusion, dispersion or fault handling, consumes only *occupancy-level* information: free, obstacle, target, crash site. Nothing in the coordination layer needs imagery, identity or appearance. A privacy-respecting embodiment would therefore process raw sensor data on-board, transiently, solely to update the occupancy and detection maps, and share *only those maps* between drones and with operators: data-protection-by-design in the sense of Article 25, implemented as an architectural boundary between a perception layer (privacy-critical, on-board, ephemeral) and the coordination layer this report describes (abstract, shareable). The gap between what the swarm *could* record and what its task *needs* is exactly the space minimization demands be closed. The decentralization cuts the other way for accountability, though: the deploying organization is plainly the controller, but processing is replicated across autonomous nodes whose knowledge states synchronize opportunistically, so retention must be enforced on every replica and an erasure request must reach state that was fused, anonymously, into multiple maps. The anonymity of fusion, a coordination virtue in Section 6.4, becomes a governance burden the moment the fused content is personal data: provenance-free knowledge cannot be selectively erased.

8.3 The EU AI Act

The AI Act [9] governs AI systems by risk tier: prohibited practices, high-risk systems subject to ex-ante requirements, and lighter transparency regimes below. A learned multi-drone search policy is unambiguously an AI system in the Act's sense; *which* tier it occupies depends, once again, on deployment. Used by or on behalf of public authorities to locate persons in a law-enforcement context, it falls within the high-risk categories of Annex III (law enforcement; similarly migration and border management), and as a safety component of critical-infrastructure inspection, high-risk again. If the target is a *person* and localization proceeds by identifying individuals from sensor data, the system approaches remote biometric identification, whose real-time use in publicly accessible spaces for law enforcement is prohibited outright (Article 5) save for narrowly enumerated exceptions, which notably include the *targeted search for missing persons*: the drafters anticipated precisely the benign reading of this capability and carved it out, subject to authorization and safeguards. A civilian system of this kind should therefore be engineered from the start to the high-risk compliance envelope.

Two of those obligations map onto this project sharply enough to be worth stating. Human oversight (Article 14) is the widest gap: the system as built has *no runtime human role whatsoever* (Section 6.8), so an oversight interface for monitoring, intervention and override would have to be added, not merely documented. Accuracy and robustness (Article 15) is the article the project speaks to most directly: the fault-robustness and generalization evaluations of Section 7 are, in substance, Article 15 evidence, while their statistical character, performance over seeded samples rather than guarantees, marks the boundary of what such evidence can establish. The remaining chapters land where one would expect, and one mapping is worth naming: for a policy trained entirely in simulation, data governance (Article 10) is a question about the randomization ranges of Section 5.1, whose limits are compliance gaps.

The military carve-out. All of the above evaporates in the scenario the next subsection considers: Article 2(3) excludes from the Act's scope systems placed on the market or used *exclusively for military, defence or national security purposes.* The European Union's flagship AI regulation, including its prohibited-practices tier, simply does not apply to a targeting swarm. The asymmetry deserves emphasis: the *rescue* deployment of this capability faces DPIAs, conformity assessments and oversight requirements; the *lethal* deployment of the same capability faces, from EU AI law, nothing. What fills the gap is the older and thinner fabric of international humanitarian law and the slow-moving intergovernmental process around autonomous weapons.

8.4 The autonomous-weapons scenario

Suppose, then, the ugly instantiation: the target is a military objective and detection cues engagement, by a human downstream or by the system itself. In the latter case the assembly meets the standard definition of an autonomous weapon system: one that, once activated, selects and engages targets without further human intervention [53]. No dedicated treaty governs such systems; the landscape consists of international humanitarian law (IHL), the CCW Group of Governmental Experts' guiding principles, under which IHL applies fully and human responsibility cannot be transferred to the machine [16], and normative positions such as the ICRC's, calling for unpredictable autonomous weapons and those targeting persons to be ruled out and the rest strictly constrained [23, 46]. Against that landscape the system described here would be an unacceptable weapon component, for specific properties established in the preceding sections rather than generic misgivings about "killer robots".

Distinction fails at the architecture's core. The target representation is a binary, monotone memory bit: once any drone marks a cell as target the mark persists, propagates through fusion to the whole connected team, and, fusion being an anonymous maximum with no provenance and no retraction, *cannot be corrected.* A single false detection becomes, irreversibly, the entire team's truth: the very monotonicity that makes cooperative mapping conflict-free makes misidentification unrecallable. Add the belief staleness of Section 6.7 and the mismatch with a legal standard

requiring case-by-case verification *at the moment of attack* is structural, not incidental.

Proportionality has no substrate. Weighing anticipated military advantage against expected incidental harm is a contextual, value-laden judgement, while the policy's entire evaluative apparatus is a Q-function distilled from the reward of Table 2: time, coverage, collision. No input channel exists through which civilian presence could even *enter* the decision. This is the goal-oriented/goal-governed distinction of Section 6.2 bearing legal weight: a system whose objectives are distilled into weights cannot deliberate about competing considerations, because nothing in it represents considerations at all.

Precaution collides with emergence, and generalization is not verification. IHL requires those who plan attacks to take constant care and to suspend when circumstances change, but the team-level behaviour of this system is emergent, shaped by training and specified nowhere. Meaningful human control, in the influential analysis the requirement that a system *track* human moral reasons and that its conduct be *traceable* to humans who understand it [45], fails on both prongs: the only "reasons" the system responds to are distilled search economics, and no human can inspect *why* the collective did what it did. Worse, the operational profile that makes autonomy militarily attractive, namely communication-denied environments, is the profile in which a human *could not* supervise even in principle: the same range limits that make centralized control unimplementable make real-time oversight unimplementable too. And the robustness claims are statistical, while an adversary is precisely an out-of-distribution generator: jamming pushes communication below anything trained, decoys exploit the irreversible target memory, spoofed beacons corrupt the belief layer the system trusts implicitly. Deploying against an adaptive opponent a policy whose guarantees are distributional is unpredictability by construction, the property the ICRC singles out as grounds for prohibition [23].

8.5 Responsibility and opacity in self-organizing systems

Beneath both regulatory analyses lies one structural problem that this report's technical sections make unusually crisp: *where, in a system like this, is the seat of responsibility?* The classical conditions, knowledge and control, are strained twice over.

They are strained *vertically* by learning: the designers chose the reward, the ranges and the architecture, but nobody chose, or can enumerate, the policy's dispositions. This is the responsibility gap in its canonical form [32], the manufacturer's control ending where the learning begins without anyone downstream gaining the control the manufacturer lost; and the opacity is not incidental polish that better documentation could remove, since a Q-network answers "why" with a number, not a trace, and with explainability went the evidential substrate that after-the-fact accountability would need.

They are strained *horizontally* by multi-agency. Outcomes are produced by several interacting policies, an environment's mediation and the accidents of who fused with whom when, and the emergent regularities of Section 6.6 belong to no drone. Moral philosophy knows this as the problem of many hands, of which this system exhibits the purest technical form, the "hands" being homogeneous, anonymous and individually thoughtless: when an emergent behaviour causes harm, operator, commander, integrator, policy trainer and reward designer each stand at a defensible distance from the outcome, and the CCW principle that responsibility cannot pass to the machine [16] names the constraint without resolving the allocation. Product-liability law patches the civil-damages corner, the revised EU directive extending strict liability to software and AI systems, including defects arising from post-deployment learning [10]; but compensation is the *floor*, not the substance, of accountability. What remains is a design obligation: if responsibility is to be attributable at all, the system must be built so that human decisions remain the traceable causes of its consequential actions, responsibility-by-design as the governance counterpart of privacy-by-design.

8.6 Design recommendations

The analysis implies a concrete design posture, most of it implementable unilaterally by the developer. Stated as commitments for any real embodiment:

- **Detection, never engagement.** The output boundary is a *location report to a human*, with no actuator capable of harm ever placed downstream of the policy: this single commitment removes the system from the autonomous-weapon category by construction and preserves the human judgement Section 8.4 showed the policy cannot supply.
- **A human on the loop, with real authority.** Consequential action requires human confirmation on *raw evidence* (imagery), not the system's binary target bit, countering the irreversible-belief failure mode, with abort authority treated as a mission constraint: where communication denial would sever oversight, the mission profile, not the oversight, is what yields (cf. Article 14).
- **Hard mission envelopes, staged autonomy.** Geofencing, duration bounds and target-class restrictions enforced *outside* the learned component, as environment-level artefacts (Sections 6.3 and 6.5) whose compliance is verifiable by construction, the ICRC's constraint-based prescription [23] in this architecture; with deployment graduating through supervised autonomy levels [44, 53], promotion gated on the evidence of Section 7 and on its honest limits.
- **Retraction-capable knowledge.** Replace the monotone target memory with a revisable representation (confidence-weighted, decaying, provenance-carrying), so a false detection can be corrected and an erasure request honoured, fixing in one stroke the distinction-critical failure of Section 8.4 and the erasure problem of Section 8.2, as the coordination-medium upgrade Section 6.10 recommended on engineering grounds.
- **Privacy-preserving perception and tamper-evident logging.** On-board, ephemeral processing of raw sensing, only occupancy-level abstractions stored, fused or transmitted, imagery retained solely on human-confirmed detection; and signed logs of observations, beliefs, fusion events and actions, sufficient to reconstruct *what each drone knew and when*, the evidential basis Section 8.5 found missing.

None of this dissolves the dual-use character diagnosed above: no design measure can prevent a determined actor from rebuilding the capability without the safeguards. What the measures achieve is narrower and still worth having, since they make the *artefact as shipped* lawful in its intended deployments, auditable when it fails, and useless as a weapon without substantial, deliberate, and therefore attributable modification.

9 Conclusions and Future Work

9.1 Conclusions

This project set out to build, analyse and evaluate a cooperative multi-agent system for a deceptively simple task, finding a hidden target on a grid, made genuinely hard by three coupled sources of uncertainty: partial observability, limited communication, and, as its defining extension, the possibility that any drone permanently breaks at any moment with no central authority to announce the loss.

What was built, and what the multi-agent reading revealed. We implemented from scratch a cooperative grid environment with sequential execution, accumulated per-drone knowledge maps, range-limited fusion, and a decentralized fault model in which crash knowledge propagates only through a passive distress beacon and ordinary communication; a single Dueling Double DQN with parameter sharing drives every drone, and per-episode domain randomization

with a compact context vector makes that one policy a generalist across team size, sensing, communication, clutter and fault rate. Read through the vocabulary of the field (Section 6), the system's most interesting properties turn out to be precisely the ones no line of code states directly. Coordination is achieved *without* protocols, performatives or a coordinator: dispersion from a shared spawn corner is driven by environment-mediated, stigmergic repulsion from swept regions; role differentiation among identical drones is broken open by a single identity scalar; and the "knowledge" each drone acts on is a first-order epistemic belief, sound but possibly stale, updated by observation and by testimony from teammates. Graceful degradation, in particular, is a system-level property that no individual drone implements, resting on redundancy and epistemic decentralization rather than on any drone knowing the true state of the team.

What the experiments and the ethical analysis showed. The evaluation (Section 7) substantiated the central claims and bounded them honestly. A single policy generalized across every randomization axis, including *beyond* its trained ranges on team size, vision and communication, driving teams of six drones it never trained on at over 90% success. Most importantly, degradation under faults is *graceful*: across the entire trained fault range success fell by under six points while the team shed, on average, more than half a drone, and even at a fault rate more than triple the trained maximum the system still completed two missions in three, with no cliff. The sweeps also separated two competences the success rate conflates: vision governs both whether and how well the team searches, while communication governs only how well, though it governs it substantially, map fusion buying close to a fifth in path quality and mission time, and a seventh in redundancy, at no cost beyond proximity. The limits were equally clear: obstacle density is the one axis whose extrapolation collapses, although a breadth-first check of the evaluation instances showed most of that collapse to be the instance distribution becoming unsolvable rather than the policy failing; and two of the three emergence signatures, together with the belief-lag question, remain defined but uninstrumented. Section 8 then argued that the capability is irreducibly dual-use, the fault tolerance that makes a search-and-rescue swarm robust being what would make an attack swarm resilient; mapped the civilian system onto the GDPR and the EU AI Act; confronted the military carve-out that lets the war scenario escape the Act entirely; and concluded that *this specific system*, opaque, statistically rather than formally validated, acting on stale beliefs and with no single decision locus, would be unacceptable as an autonomous weapon, on exactly the grounds its own evaluation exposes. The takeaway is not that the technology should not be built, but that its governance cannot be an afterthought to its engineering.

9.2 Future work

Three measurements were defined in this report but not carried out, each for a mundane reason. The belief-lag question, that is how communication range governs the interval between a fault and a survivor's discovery of it, needs a joint fault×comm sweep with per-step trajectory logging, and the same logging would deliver the two emergence signatures (inter-drone distance over time, post-fault re-covering latency) that Section 7.6 left qualitative; two further ablations, removing the believed-alive count from the context vector and switching the shared terminal reward for an individual one, require retraining a policy each. None of these changes the architecture: they are instrumentation and compute, and they would sharpen the claims already made.

Three directions reach further. On coordination, the finding that communication buys efficiency but not find rate here is partly a property of the task and partly of the design: *learned* messages would let the system decide *what* to share, and a task that genuinely rewarded information sharing (a target visible only from afar, heterogeneous sensors, partial views that must be combined) would exercise that channel far harder than target-in-a-maze search does, while value-decomposition methods (VDN/QMIX-style) would replace the shared-terminal credit heuristic with a learned factorization of team value. On representation, the hand-crafted knowledge maps that let a memoryless policy act under partial observability could give way to a learned recurrent or attention-based memory over the observation history. And on the world itself, every limitation

of Section 7.7 is a research direction: moving or evasive targets, correlated or passage-blocking faults, and, most pointedly given the ethical analysis, an *adversary* choosing when and whom to disable and how to jam communication, turning the benign i.i.d. fault process into a strategic one and the cooperative game into a mixed one. Closing the sim-to-real gap while preserving the fault tolerance that is the point of the exercise is the direction in which the work would have to move to matter outside the laboratory.

A Configuration and Hyperparameters

All experiments are driven by a single configuration file (`configs/default.yaml`), the single source of truth referenced throughout Sections 4–5. The tables below reproduce the values used for the final run reported in this document. The reward terms are not repeated here; they are given in Table 2.

Parameter	**Value**	**Notes**
Grid size	32×32	fixed; never randomized
Action space	4 (up/down/left/right)	no "stay" action; masked at grid/obstacle edges
Vision metric	Chebyshev	$(2r_{\text{vis}}{+}1)^2$ square window
Comm. metric	Manhattan	range-limited pairwise map fusion
Global channels	6	Visited, Obstacle, Trajectory, Target, Own_Pos., Broken
Local channels	5	cropped 13×13 egocentric patch
Context dimension	5	$[r_{\text{vis}}, r_{\text{comm}}, N, \text{agent\_id}, \hat{n}]$
Discount γ	0.97	effective horizon for search

Table 6: Environment and observation constants.

Factor	**Range (final run)**	**Normalization denom.**
Vision radius r_{vis}	$[1, 6]$ (integer)	6
Comm. range r_{comm}	$[2, 12]$ (integer)	12
Team size N	$[1, 4]$ (integer)	4 (also used for agent_id, $\hat{n}$)
Obstacle density ρ	$[0.10, 0.30]$ (float)	environment-level (not a context entry)
Fault prob. p_{fault}	$[0.0, 0.003]$ (float)	environment-level (not a context entry)

Table 7: Per-episode domain-randomization ranges, sampled uniformly at every reset (Section 5.1). Context factors are normalized by their maxima (Section 4.5); density and fault probability are deliberately *not* exposed to the network.

Phase	**Episodes**	**Overrides**
`test`	0 (skipped)	staging placeholder; narrower `dr_override`
`mas_50k_dr_faults`	50,000	`max_steps`=200, `n_agents`=4 (buffer sizing)

Table 8: Curriculum. The final policy is trained from scratch in the single main phase; the zero-episode debug phase is kept only for staging (Section 5.1). The phase's `n_agents` sets the initial allocation and replay-buffer width; team size is still resampled in $[1, 4]$ every episode from the ranges of Table 7.

Hyperparameter	Value	Notes
Optimizer	Adam	weight decay 10^{-6}
Learning rate	$10^{-4} \to 5 \times 10^{-4} \to 10^{-6}$	linear warmup (10%) then cosine; per episode
Gradient clip	1.0	global norm
Discount γ	0.97	—
n-step return	3	per-agent windows; team credit needs $n \geq 2$
Loss	Huber (smooth L_1)	Double-DQN target
Replay buffer	2×10^5	uniform; not serialized in checkpoints
Batch size	16	—
Update cadence	every 4 agent-turns	one gradient step
Target sync	every 500 gradient steps	hard copy
ε schedule	$1.0 \to 0.05$, $\times 0.9995$/episode	floor at ep. 5,990
FC head widths	$1536 \to 768 \to 384$	dueling head $Q = V + A - \overline{A}$

Table 9: Agent and training hyperparameters (Sections 4.4, 4.6, 5.2). Network normalization is LayerNorm/GRN only, for batch-size-independent inference.

Setting	Value	Purpose
In-training eval interval	every 1,000 episodes	checkpoint selection
In-training eval instances	20 fixed seeds	success-dominant rule
In-training eval ε	0.05	near-greedy
Periodic checkpoint save	every 2,500 episodes	learning-curve axis
Parallel watcher eval	200 seeded episodes/checkpoint	crash-safe second opinion
OFAT sweep seeds	500 per point	Section 7
Learning-curve / heatmap seeds	300 per point	idem

Table 10: Evaluation protocol (Sections 5.4, 7.1).

B Repository Guide

The code is organized as a small set of packages behind three entry points.

Module map.

- `env/`: the environment package, with `grid_env.py` (the `DroneSearchEnv`: generation with BFS reachability, sequential `step_agent`, reward, beacon detection, communication fusion) and `utils.py` (BFS helpers and the eval-only `auto_nav_action`).
- `agents/`: `networks.py` (GlobalMapCNN, LocalCNN, dueling head), `dqn_agent.py` (Double-DQN agent: action selection, learning step, checkpoint I/O), `nstep.py` (per-drone n-step windows), `replay_buffer.py`, `scheduler_utils.py` (warmup + cosine LR) and `base_agent.py`.
- `training/`: `train.py` (curriculum loop, domain randomization, n-step windows, in-training evaluation).
- `gui/`: the renderer for the `play`, `eval` and `simulate` modes (fault slider, auto-nav toggle).
- `testing/`: `evaluate_policy.py`, the OFAT, fault, heatmap and epoch harness, `analysis.ipynb` (every figure of Section 7) and `make_training_figures.py` (the telemetry and schedule figures of Section 5).
- `demo/`: the code that renders the narrated video walkthrough end to end from the trained policy and the experiment CSVs; `hpc/`: the SLURM batch scripts that produced the results.
- Top level: `main.py` (all runtime modes), `eval_checkpoints.py` (batch and `-watch` checkpoint ranking), and `configs/default.yaml` (single source of truth).

Trained weights. Every number in Section 7 comes from the final checkpoint of the 50,000-episode run, `mas_50k_dr_faults_ep50000.pt`. It is too large for the repository and is published separately:

https://huggingface.co/simoswish/MAS_MultiDroneExploration

Place it under `checkpoints/` and pass it to `-checkpoint` (written `$CKPT` below); the `q_net` entry of the saved dictionary holds the network weights.

Environment setup.

```
conda env create -f environment.yaml   # env "rl-drone": python 3.11, CUDA 11.8
conda activate rl-drone
# or, without conda:  pip install -r requirements.txt
```

Reproducing the results.

```
# Training (curriculum from configs/default.yaml; --resume to continue)
python main.py --mode train

# Qualitative inspection (CKPT = the published checkpoint above)
python main.py --mode eval --checkpoint $CKPT [--fault-prob P] [--auto-nav]
python main.py --mode simulate            # GUI: fault slider, auto-nav toggle

# The experiments of Section 7 (writes testing_results/{csv,plots})
python testing/evaluate_policy.py --axis all      --seeds 500
python testing/evaluate_policy.py --axis fault    --seeds 500 --auto-nav
python testing/evaluate_policy.py --axis heatmaps --seeds 300
python testing/evaluate_policy.py --axis epoch    --seeds 300
# Every figure in this report ("make figures")
jupyter nbconvert --to notebook --execute --inplace testing/analysis.ipynb
python testing/make_training_figures.py
```

C Additional Figures

For completeness this appendix collects two figures that complement the body. Figure 14 gives the step-budget sweep summarized in Table 4, with the six panels following the layout of Figures 6 and 8, and Figure 15 documents the two per-episode schedules of the training run of Section 5.

References

[1] Peter Anderson, Angel Chang, Devendra Singh Chaplot, Alexey Dosovitskiy, Saurabh Gupta, Vladlen Koltun, Jana Kosecka, Jitendra Malik, Roozbeh Mottaghi, Manolis Savva, and Amir R. Zamir. On evaluation of embodied navigation agents. Technical Report arXiv:1807.06757, arXiv, 2018.

[2] W. Ross Ashby. Principles of the self-organizing system. In *Principles of Self-Organization*, pages 255–278. Pergamon Press, 1962.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[4] Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4): 819–840, 2002.

[5] Eric Bonabeau. Agent-based modeling: Methods and techniques for simulating human systems. *Proceedings of the National Academy of Sciences*, 99(suppl. 3):7280–7287, 2002.

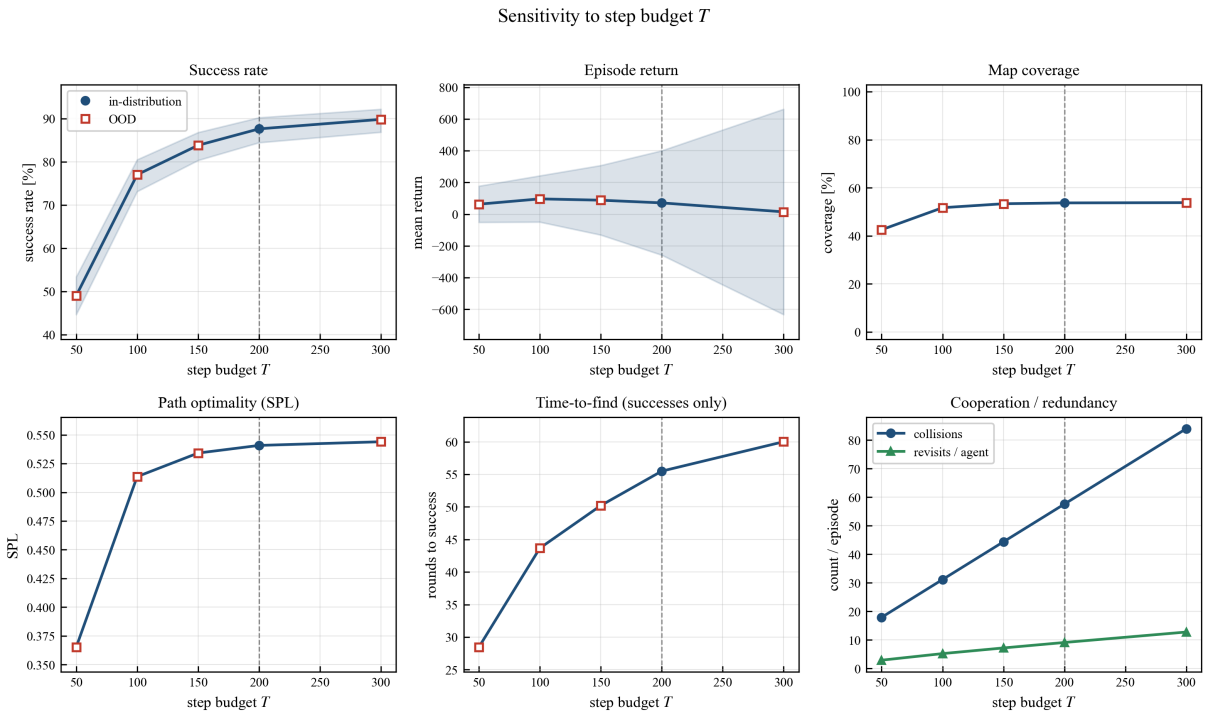

Figure 14: Sensitivity to the step budget T. Success rises with diminishing returns as the budget grows; a larger budget only rescues episodes that were already close to succeeding. The budget was never randomized, training having fixed $T = 200$: the dashed line marks that single trained value, and every other point is an extrapolation, so the white/shaded convention of Section 7 does not apply on this axis.

[6] Michael E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, 1987.

[7] Scott Camazine, Jean-Louis Deneubourg, Nigel R. Franks, James Sneyd, Guy Theraulaz, and Eric Bonabeau. *Self-Organization in Biological Systems*. Princeton University Press, 2001.

[8] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation). Official Journal of the European Union, L 119, 2016.

[9] European Parliament and Council. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). Official Journal of the European Union, L series, 2024.

[10] European Parliament and Council. Directive (EU) 2024/2853 on liability for defective products. Official Journal of the European Union, L series, 2024.

[11] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the 3rd International Conference on Information and Knowledge Management (CIKM)*, pages 456–463, 1994.

[12] Stan Franklin and Art Graesser. Is it an agent, or just a program?: A taxonomy for autonomous agents. In *Intelligent Agents III: Agent Theories, Architectures, and Languages (ATAL 1996), LNCS 1193*, pages 21–35. Springer, 1997.

[13] David Gelernter. Generative communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985.

[14] Jeffrey Goldstein. Emergence as a construct: History and issues. *Emergence*, 1(1):49–72, 1999.

[15] Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles chez *Bellicositermes natalensis* et *Cubitermes* sp. la théorie de la stigmergie. *Insectes Sociaux*, 6(1):41–80, 1959.

[16] Group of Governmental Experts on Lethal Autonomous Weapons Systems. Guiding principles affirmed by the GGE on emerging technologies in the area of LAWS. CCW/MSP/2019/9, United Nations, 2019.

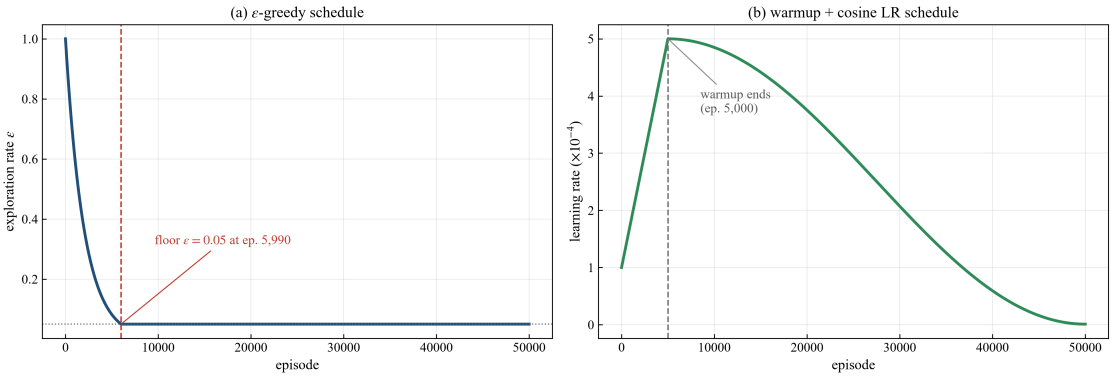

Figure 15: The two per-episode schedules of the final 50,000-episode run. *(a)* ε decays multiplicatively to its 0.05 floor at episode 5,990; *(b)* the learning rate warms up over the first 10% of episodes to 5×10^{-4}, then anneals cosine-wise to 10^{-6}. The two are deliberately out of phase: exploration has essentially finished annealing while the learning rate is still near its peak, so the bulk of the high-learning-rate budget is spent on a near-greedy, and therefore task-relevant, state distribution.

[17] Jayesh K. Gupta, Maxim Egorov, and Mykel Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *Autonomous Agents and Multiagent Systems (AAMAS 2017 Workshops)*, pages 66–83, 2017.

[18] Assaf Hallak, Dotan Di Castro, and Shie Mannor. Contextual Markov decision processes. arXiv preprint arXiv:1502.02259, 2015.

[19] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[20] Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6):750–797, 2019.

[21] Matteo Hessel, Joseph Modayil, Hado van Hasselt, et al. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 3215–3222, 2018.

[22] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, pages 646–661, 2016.

[23] International Committee of the Red Cross. ICRC position on autonomous weapon systems. Geneva, 12 May 2021, 2021.

[24] Nicholas R. Jennings. An agent-based approach for building complex software systems. *Communications of the ACM*, 44(4):35–41, 2001.

[25] Kyle D. Julian and Mykel J. Kochenderfer. Distributed wildfire surveillance with autonomous aircraft using deep reinforcement learning. *Journal of Guidance, Control, and Dynamics*, 42(8):1768–1778, 2019.

[26] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[27] Landon Kraemer and Bikramjit Banerjee. Multi-agent reinforcement learning as a rehearsal for decentralized planning. *Neurocomputing*, 190:82–94, 2016.

[28] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.

[29] Michael L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the 11th International Conference on Machine Learning*, pages 157–163, 1994.

[30] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11986, 2022.

[31] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems 30*, pages 6379–6390, 2017.

[32] Andreas Matthias. The responsibility gap: Ascribing responsibility for the actions of learning automata. *Ethics and Information Technology*, 6(3):175–183, 2004.

[33] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[34] H. Penny Nii. The blackboard model of problem solving and the evolution of blackboard architectures. *AI Magazine*, 7(2):38–53, 1986.

[35] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.

[36] Andrea Omicini and Enrico Denti. From tuple spaces to tuple centres. *Science of Computer Programming*, 41(3):277–294, 2001.

[37] Andrea Omicini, Alessandro Ricci, and Mirko Viroli. Artifacts in the A&A meta-model for multi-agent systems. *Autonomous Agents and Multi-Agent Systems*, 17(3):432–456, 2008.

[38] Fabio Pardo, Arash Tavakoli, Vitaly Levdik, and Petar Kormushev. Time limits in reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, pages 4045–4054, 2018.

[39] H. Van Dyke Parunak. "Go to the ant": Engineering principles from natural multi-agent systems. *Annals of Operations Research*, 75:69–101, 1997.

[40] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018.

[41] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 3942–3951, 2018.

[42] Jorge Peña Queralta, Jussi Taipalmaa, Bilge Can Pullinen, Victor Kathan Sarker, Tuan Nguyen Gia, Hannu Tenhunen, Moncef Gabbouj, Jenni Raitoharju, and Tomi Westerlund. Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision. *IEEE Access*, 8: 191617–191643, 2020.

[43] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the 1st International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995.

[44] SAE International. Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles (SAE J3016). Standard J3016_202104, 2021.

[45] Filippo Santoni de Sio and Jeroen van den Hoven. Meaningful human control over autonomous systems: A philosophical account. *Frontiers in Robotics and AI*, 5:15, 2018.

[46] Noel Sharkey. Autonomous warfare. *Scientific American*, 322(2):52–57, 2020.

[47] Aravind Srinivas, Tsung-Yi Lin, Niki Parmar, Jonathon Shlens, Pieter Abbeel, and Ashish Vaswani. Bottleneck transformers for visual recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16519–16529, 2021.

[48] Lucy A. Suchman. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987.

[49] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[50] Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the 10th International Conference on Machine Learning*, pages 330–337, 1993.

[51] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.

[52] Mark Towers, Ariel Kwiatkowski, Jordan Terry, et al. Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032, 2024.

[53] U.S. Department of Defense. Directive 3000.09: Autonomy in weapon systems. Washington, DC (updated 2023), 2012.

[54] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.

[55] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.

[56] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1995–2003, 2016.

[57] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[58] Danny Weyns, Andrea Omicini, and James Odell. Environment as a first class abstraction in multiagent systems. *Autonomous Agents and Multi-Agent Systems*, 14(1):5–30, 2007.

[59] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16133–16142, 2023.

[60] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

[61] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.

[62] Franco Zambonelli and H. Van Dyke Parunak. Signs of a revolution in computer science and software engineering. *Engineering Societies in the Agents World III (ESAW 2002), LNCS 2577*, pages 13–28, 2003.